# 模板

## 图论

### 割点

```
vi G[MAXN];
int dfn[MAXN] , low[MAXN] , clo , cut[MAXN];
void tarjan( int u , int fa ) {
    int chi = 0;
    dfn[u] = low[u] = ++ clo;
    for( int v : G[u] ) {
        if( !dfn[v] ) {
            tarjan( v , u );
            low[u] = min( low[u] , low[v] );
            if( u != fa && low[v] >= dfn[u] ) cut[u] = 1;
            if( u == fa ) ++ chi;
        }
        low[u] = min( low[u] , dfn[v] );
    }
    if( u == fa && chi > 1 ) cut[u] = 1;
}

void solve() {
    cin >> n >> m;
    rep( i , 1 , m ) {
        static int u , v;
        scanf("%d%d",&u,&v);
        G[u].pb( v ) , G[v].pb( u );
    }
    rep( i , 1 , n ) if( !dfn[i] ) {
        tarjan( i , i );
    }
    int sz = 0;
    rep( i , 1 , n ) if( cut[i] ) ++ sz;
    cout << sz << endl;
    rep( i , 1 , n ) if( cut[i] ) printf("%d ",i);
}
```

### 圆方树

```
vi G[MAXN];
int cnt;

vi g[MAXN];
int dfn[MAXN] , low[MAXN] , clo;
int stk[MAXN] , tp;
multiset<int> S[MAXN];
void tarjan( int u ) {
    dfn[u] = low[u] = ++ clo;
    stk[++ tp] = u;
    for( int v : g[u] ) {
```

```
        if( !dfn[v] ) {
            tarjan( v );
            low[u] = min( low[u] , low[v] );
            if( low[v] == dfn[u] ) {
                ++ cnt;
//              cout << u << ' ' << cnt << endl;
                G[u].pb( cnt );
                int x;
                do {
                    x = stk[tp];
//                  cout << cnt << ' ' << x << endl;
                    G[cnt].pb( x );
                    S[cnt].insert( A[x] );
                    -- tp;
                } while( x != v );
            }
        } else low[u] = min( low[u] , dfn[v] );
    }
}
```

## 最大流

```
int n , m , s , t;

int head[MAXN] , to[MAXN << 1] , nex[MAXN << 1] , wto[MAXN << 1] , ecn = -1;
void ade( int u , int v , int w ) {
    to[++ ecn] = v , nex[ecn] = head[u] , wto[ecn] = w , head[u] = ecn;
    to[++ ecn] = u , nex[ecn] = head[v] , wto[ecn] = 0 , head[v] = ecn;
}

queue<int> Q;
int dep[MAXN] , cur[MAXN];
const int inf = 1e9;
bool bfs( ) {
    Q.push( s );
    rep( i , 1 , n ) cur[i] = head[i];
    memset( dep , 0x3f , sizeof dep );
    dep[s] = 0;
    while( !Q.empty() ) {
        int u = Q.front(); Q.pop();
        for( int i = head[u] ; ~i ; i = nex[i] ) if( dep[to[i]] > inf && wto[i] )
                dep[to[i]] = dep[u] + 1 , Q.push( to[i] );
    }
    return dep[t] < inf;
}

ll dfs( int u , ll lim ) {
    if( !lim || u == t ) return lim;
    ll flow = 0 , f;
    for( int& i = cur[u] ; ~i ; i = nex[i] ) if( dep[to[i]] == dep[u] + 1 && ( f
= dfs( to[i] , min( lim , 1ll * wto[i] ) ) ) ) {
            lim -= f , flow += f , wto[i] -= f , wto[i ^ 1] += f;
            if( !lim ) break;
        }
    return flow;
```

```
}

ll dinic( ) {
    ll res = 0;
    while( bfs( ) )
        res += dfs( s , 0x3f3f3f3f3f3f3f3f );
    return res;
}

void solve() {
    memset( head , -1 , sizeof head );
    cin >> n >> m >> s >> t;
    int u , v , w;
    rep( i , 1 , m ) {
        scanf("%d%d%d",&u,&v,&w);
        ade( u , v , w );
    }
    cout << dinic( ) << endl;
}
```

## 费用流

```
int n , m , s , t;

int head[MAXN] , to[MAXN << 1] , nex[MAXN << 1] , wto[MAXN << 1] , cto[MAXN << 1]
, ecn = -1;
void ade( int u , int v , int w , int c ) {
    to[++ ecn] = v , nex[ecn] = head[u] , wto[ecn] = w , head[u] = ecn , cto[ecn]
= c;
    to[++ ecn] = u , nex[ecn] = head[v] , wto[ecn] = 0 , head[v] = ecn , cto[ecn]
= -c;
}
queue<int> Q;
int dis[MAXN] , cur[MAXN] , vis[MAXN];
const int inf = 1e9;
bool spfa( int s ) {
    mem( vis );
    Q.push( s );
    memset( dis , 0x3f , sizeof dis );
    dis[s] = 0 , vis[s] = 1;
    rep( i , 1 , n ) cur[i] = head[i];
    while( !Q.empty() ) {
        int u = Q.front(); Q.pop();
        vis[u] = 0;
        for( int i = head[u] ; ~i ; i = nex[i] ) if( wto[i] && dis[to[i]] >
dis[u] + cto[i] ) {
            int v = to[i];
            dis[v] = dis[u] + cto[i];
            if( !vis[v] ) Q.push( v ) , vis[v] = 1;
        }
    }
    return dis[t] < inf;
}
int co;
int dfs( int u , int lim ) {
```

```
    if( !lim || u == t ) return lim;
    vis[u] = 1;
    int flow = 0 , f;
    for( int& i = cur[u] ; ~i ; i = nex[i] ) if( !vis[to[i]] && wto[i] &&
dis[to[i]] == dis[u] + cto[i] && ( f = dfs( to[i] , min( lim , wto[i] ) ) ) ) {
        lim -= f , flow += f , wto[i] -= f , wto[i ^ 1] += f;
        co += f * cto[i];
        if( !lim ) break;
    }
    vis[u] = 0;
    return flow;
}
int mcmf( ) {
    int re = 0 , x;
    while( spfa( s ) ) {
        re += dfs( s , 0x3f3f3f3f );
    }
    return re;
}

void solve() {
    memset( head , -1 , sizeof head );
    cin >> n >> m >> s >> t;
    int u , v , w , c;
    rep( i , 1 , m ) {
        scanf("%d%d%d%d",&u,&v,&w,&c);
        ade( u , v , w , c );
    }
    cout << mcmf( ) << ' ';
    cout << co << endl;
}
```

## 上下界流

不知道当时从哪里粘的

虽然题目要求有源汇有上下界的最大流，但先让我们考虑**无源汇有上下界可行流**。

## 无源汇有上下界可行流

它的定义非常简单：一张$n$个点$m$条边网络流，其中每条边有一个流量限制$[L_i, R_i]$表示这条边**流量的范围**。因为没有源汇，所以这张网络流需要满足**流入每个点的流量总量=流出的流量总量**。

既然每个点都有上下界，那么我们先满足它的下界好了。即**对于每条边先填充$\mathbf{L_i}$的流量**。但是这样显然流量不平衡，也就是说，有的点凭空流出了流量，但是又有流量流入点之后消失了。不过，我们可以惊喜的发现，问题已经转化为了：

每个点有一个权值$\mathbf{W_i}$表示当前在这个点上**积蓄/欠缺的流量**，同时每条边有一个流量限制$[\mathbf{0}, \mathbf{R_i} - \mathbf{L_i}]$。现在要你求出一种方案，使得每个点积蓄/欠缺的流量全部用完，并且每条边满足流量限制。

如果我们再改一下题面：有一个超级源点$S$向满足$W_i > 0$的点连了一条容量为$|W_i|$的边。同时又有一个超级汇点$T$使得所有满足$W_i < 0$的边都有一条容量为$|W_i|$连向了$T$。这条题目是不是非常像有源汇的网络流最大流？

**让我们重新梳理一下上述内容**:我们强制给每条边添上了一些流，或者说将这条边的流量范围变为了$[0, R_i - L_i]$的普通版最大流。这样做会造成一些点上多出或欠缺一些流量。这时我们将欠的流量当作是$S$给他们的流量，多出来的流量要还给$T$，我们就能用最大流给他们疏通疏通经脉，满足约束条件。**此时我们能够使得每条边满足约束条件并且流量平衡。**

因为对于每条边$(u, v, w)$，我们都让它的起始点$u$"透支"了$w$的流量，而给终止点$v$"借了"$w$流量，因此整张网络上的流量总和为$0$。

但是先别高兴，我们要解决的是**有源汇有上下界可行流**。

## 有源汇有上下界可行流

**有源汇**这个条件给了我们什么东西呢？他给了我们源点$s$和汇点$t$，其中$s$只输出流量，而$t$只接受流量。对于这两个点，我们不要求它们流入的量=流出的量。但是如何把它转化为无源汇的上下界可行流呢？

其实，我们可以通过连接一条边$t \to s$，容量为$[0, +\infty]$，这样就能使得$S$流出的流量由$T$提供，$T$流入的量由$S$提供，进而达到收支平衡。这样子跑一遍无源汇有上下界可行流，就可以使得到一组可行解了。其中，在$t \to s$这条边上**流过的**流量就是一个可行的方案的答案了。换句话说，在$t \to s$这条边流过的流量就是它的反向边（即残量网络的对应边）的流量。

## 有源汇有上下界最大流

但是我们的问题是要求最大流呀！不要慌，至少我们已经找到了一种合法的情况了。最大流算法需要扩大答案怎么办？在残量网络上跑增广路。而有源汇有上下界最大流也不例外（因为我们实际上已经把问题转化为了最大流问题，不过是多出了两个特殊点$S$和$T$）。如果我们从$s$向$t$增广会发生什么呢？

让我们重新回忆一下增广路的概念：从点$s$到点$t$增广，就是尽可能地在残量网络中寻找一条$s \to t$的道路并转化为$s \to t$的流量。

由于$S$没有入边，而$T$没有出边，因此增广路不会经过$S, T$，也就不会导致约束条件失效（即始终满足流量平衡）。

## 建图

说了那么多，应该回到题目了。记$D'_i$表示每一天对应的点，$G'_i$为每一位少女对应的点。首先我们需要建立一个源点$s$，由于每一天拍摄的照片数量不超过$D_i$，因此从$s$向$D_i$连上约束为$[0, D_i]$ 的边。对于第$i$天，我们从该天向少女$T_{i,j}$连上边$D'_i \to G'_{T_{i,j}}$，约束条件为$[L_{i,j}, R_{i,j}]$。最后，每名少女需要向汇点$T$连边$G'_i \to T$，约束条件为$[G_i, +\infty]$。跑一遍有源汇有上下界最大流即可。

**总结**：这条题目的流程为读入边，处理边，跑普通版最大流。因此编写难度并不是很大。

```
struct Node {
    struct Edge *firstEdge, *currEdge;
    long long level, extraIn;
} N[MAXN + 2];

struct Edge {
    Node *from, *to;
    int cap, flow;
    Edge *next, *revEdge;

    Edge(Node *from, Node *to, int cap) : from(from), to(to), cap(cap), flow(0),
next(from->firstEdge) {}
};

struct Dinic {
    bool makeLevelGraph(Node *s, Node *t, int n) {
        for (int i = 0; i < n; i++) {
            N[i].level = 0;
            N[i].currEdge = N[i].firstEdge;
        }

        std::queue<Node *> q;
        q.push(s);

        s->level = 1;

        while (!q.empty()) {
            Node *v = q.front();
            q.pop();

            for (Edge *e = v->firstEdge; e; e = e->next) {
                if (e->flow < e->cap && !e->to->level) {
                    e->to->level = v->level + 1;
                    if (e->to == t)
                        return true;
                    else
                        q.push(e->to);
                }
            }
        }

        return false;
    }

    int findPath(Node *s, Node *t, int limit = INT_MAX) {
        if (s == t)
            return limit;

        for (Edge *&e = s->currEdge; e; e = e->next) {
            if (e->to->level == s->level + 1 && e->flow < e->cap) {
                int flow = findPath(e->to, t, std::min(limit, e->cap - e->flow));
                if (flow) {
                    e->flow += flow;
                    e->revEdge->flow -= flow;
                    return flow;
                }
```

```
            }
        }

        return 0;
    }

    int operator()(int s, int t, int n) {
        int res = 0;
        while (makeLevelGraph(&N[s], &N[t], n)) {
            int flow;
            while ((flow = findPath(&N[s], &N[t])) > 0) res += flow;
        }

        return res;
    }
} dinic;

inline Edge* addEdge(int from, int to, int cap) {
    N[from].firstEdge = new Edge(&N[from], &N[to], cap);
    Edge* ret = N[from].firstEdge;
    N[to].firstEdge = new Edge(&N[to], &N[from], 0);

    N[from].firstEdge->revEdge = N[to].firstEdge;
    N[to].firstEdge->revEdge = N[from].firstEdge;
    return ret;
}

inline Edge* addEdge(int from, int to, int lower, int upper) {
//    printf("%d %d %d %d\n",from,to,lower,upper);
    int cap = upper - lower;
    Edge* ret = addEdge(from, to, cap);

    N[to].extraIn += lower;
    N[from].extraIn -= lower;
    return ret;
}
```

## 2-sat

```
vi G[MAXN];
void ade( int u , int v ) {
    G[u].pb( v );
}
int dfn[MAXN] , low[MAXN] , ins[MAXN] , stk[MAXN] , top , cnt , bel[MAXN] , scc;
void tarjan( int u ) {
    stk[++ top] = u , ins[u] = 1 , dfn[u] = low[u] = ++ cnt;
    for( int v : G[u] ) {
        if( !dfn[v] ) tarjan( v ) , low[u] = min( low[u] , low[v] );
        else if( ins[v] ) low[u] = min( low[u] , dfn[v] );
    }
    if( low[u] == dfn[u] ) {
        ++ scc;
        int x;
        do {
            x = stk[top--];
```

```
            bel[x] = scc , ins[x] = 0;
        } while( x != u );
    }
}

void solve() {
    cin >> n >> m;
    int i , a , j , b , u , v;
    rep( k , 1 , m ) {
        scanf("%d%d%d%d",&i,&a,&j,&b);
        ade( ( i - 1 ) << 1 | (!a) , ( j - 1 ) << 1 | b );
        ade( ( j - 1 ) << 1 | (!b) , ( i - 1 ) << 1 | a );
    }
    rep( i , 0 , 2 * n - 1 ) if( !dfn[i] ) tarjan( i );
    vi ans;
    rep( i , 1 , n ) {
        int t0 = ( i - 1 ) << 1 , t1 = ( i - 1 ) << 1 | 1;
        if( bel[t0] == bel[t1] ) return void( puts("IMPOSSIBLE") );
        else if( bel[t0] < bel[t1] ) ans.pb( 0 );
        else ans.pb( 1 );
    }
    puts("POSSIBLE");
    for( int x : ans ) printf("%d ",x);
}
```

用于写模拟2sat的:

```
const int N = 1e6 + 5;
int n, m, co[N * 2], stk[N * 2], tp;
vector<int> G[N * 2];
int dfs(int u) {
    if(co[u ^ 1]) return 0;
    if(co[u]) return 1;
    co[u] = 1, stk[++tp] = u;
    for(int v : G[u]) if(!dfs(v)) return 0;
    return 1;
}
int twoSat() {
    int u, a, v, b;
    while(m--) {
        scanf("%d%d%d%d", &u, &a, &v, &b);
        G[u * 2 + !a].pb(v * 2 + b);
        G[v * 2 + !b].pb(u * 2 + a);
    }
    rep(i, 1, n) {
        if(!co[i * 2] && !co[i * 2 + 1] && !dfs(i * 2)) {
            while(tp) co[stk[tp--]] = 0;
            if(!dfs(i * 2 + 1)) return 0;
        }
        tp = 0;
    }
    return 1;
}
int main() {
    cin >> n >> m;
```

```
    if(twoSat()) {
        puts("POSSIBLE");
        rep(i, 1, n) printf("%d ", !co[i * 2]);
    } else puts("IMPOSSIBLE");
    return 0;
}
```

## 负环

```
int head[MAXN] , to[MAXN << 1] , nex[MAXN << 1] , wto[MAXN << 1] , ecn;
void ade( int u , int v , int w ) {
    to[++ ecn] = v , nex[ecn] = head[u] , wto[ecn] = w , head[u] = ecn;
}
queue<int> Q;
int tim[MAXN] , dis[MAXN] , vis[MAXN];
bool spfa( int s ) {
    while( !Q.empty() ) Q.pop();
    Q.push( s ) , vis[s] = 1 , dis[s] = 0;
    while( !Q.empty( ) ) {
        int u = Q.front( ); Q.pop();
        //cout << u << endl;
        vis[u] = 0;
        for( int i = head[u] ; i ; i = nex[i] ) {
            int v = to[i];
            if( dis[v] > dis[u] + wto[i] ) {
                dis[v] = dis[u] + wto[i];
                if( !vis[v] )
                    ++ tim[v] , Q.push( v ) , vis[v] = 1;
                if( tim[v] > n ) return 0;
            }
        }
    }
    return 1;
}
```

## 最小斯坦纳树

花费最小的代价连通给定的 $k$ 个点。$O(n3^k + 2^k m \log m)$ 。

```
int n , m , k;
vector<pii> G[MAXN];
int dp[MAXN][1 << 12];

queue<int> Q;
int vis[MAXN];
void spfa( int st ) {
    while( !Q.empty() ) {
        int u = Q.front(); Q.pop();
        vis[u] = 0;
        for( auto& t : G[u] ) {
            int v = t.fi;
            if( dp[v][st] > dp[u][st] + t.se ) {
                dp[v][st] = dp[u][st] + t.se;
                if( !vis[v] ) vis[v] = 1 , Q.push( v );
```

```
            }
        }
    }
}

void solve() {
    cin >> n >> m >> k;
    rep( i , 1 , m ) {
        static int u , v , w;
        scanf("%d%d%d",&u,&v,&w);
        G[u].eb( mp( v , w ) ) , G[v].eb( mp( u , w ) );
    }
    memset( dp , 0x3f , sizeof dp );
    rep( i , 1 , k ) {
        static int u;
        scanf("%d",&u);
        dp[u][1 << i - 1] = 0;
    }
    rep( st , 1 , ( 1 << k ) - 1 ) {
        rep( i , 1 , n ) {
            for( int j = ( st - 1 ) & st ; j ; j = ( j - 1 ) & st )
                dp[i][st] = min( dp[i][st] , dp[i][j] + dp[i][j ^ st] );
            if( dp[i][st] < 1e9 ) Q.push( i ) , vis[i] = 1;
        }
        spfa( st );
    }
    int as = 0x3f3f3f3f;
    rep( i , 1 , n ) as = min( as , dp[i][( 1 << k ) - 1] );
    printf("%d\n",as);
}
```

# 字符串

## KMP

```
int n , m;
int nxt[MAXN];
void getnex( char* O , int n ) {
    int i = 0 , j = -1;
    nxt[0] = -1;
    while( i < n ) {
        if( j == -1 || O[i] == O[j] ) {
            ++ i , ++ j;
            nxt[i] = j;
        } else j = nxt[j];
    }
}
void kmp( char* S , char* T ) {
    int i = 0 , j = 0;
    while( i < n ) {
        if( j == -1 || S[i] == T[j] ) {
            ++ i , ++ j;
            if( j == m ) printf("%d\n",i - m + 1);
        } else j = nxt[j];
```

```
    }
}
```

## EXKMP

1. Z: 串与自己的每个后缀的 LCP
2. 串与另一个串的每个后缀的 LCP

```
const int N = 2e7 + 7;
int n, m, z[N], p[N];
char a[N], b[N];

inline void Z(char *s, int n) {
    for (int i = 1; i <= n; i++) z[i] = 0;
    z[1] = n;
    for (int i = 2, l = 0, r = 0; i <= n; i++) {
        if (i <= r) z[i] = min(z[i-l+1], r - i + 1);
        while (i + z[i] <= n && s[i+z[i]] == s[z[i]+1]) ++z[i];
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
}

inline void exkmp(char *s, int n, char *t, int m) {
    Z(t, m);
    for (int i = 1; i <= n; i++) p[i] = 0;
    for (int i = 1, l = 0, r = 0; i <= n; i++) {
        if (i <= r) p[i] = min(z[i-l+1], r - i + 1);
        while (i + p[i] <= n && s[i+p[i]] == t[p[i]+1]) ++p[i];
        if (i + p[i] - 1 > r) l = i, r = i + p[i] - 1;
    }
}

int main() {
    rds(a, n), rds(b, m);
    exkmp(a, n, b, m);
    ll ans = 0;
    for (int i = 1; i <= m; i++) ans ^= 1ll * i * (z[i] + 1);
    print(ans);
    ans = 0;
    for (int i = 1; i <= n; i++) ans ^= 1ll * i * (p[i] + 1);
    print(ans);
    return 0;
}
```

## 最小表示法

```
void solve() {
    cin >> n;
    rep( i , 1 , n ) scanf("%d",A + i);
    int a = 1 , b = 2 , k = 0;
    while( a <= n && b <= n && k < n ) {
        int na = ( a + k - 1 ) % n + 1 , nb = ( b + k - 1 ) % n + 1;
        if( A[na] == A[nb] ) ++ k;
```

```
        else {
            if( A[na] > A[nb] ) a += k + 1;
            else b += k + 1;
            if( a == b ) ++ a;
            k = 0;
        }
    }
    int as = min( a , b );
    vi pr;
    rep( i , as , n ) pr.pb( A[i] );
    rep( i , 1 , as - 1 ) pr.pb( A[i] );
    for( int x : pr ) printf("%d ",x);
}
```

## ACAM

```
int son[MAXN][26] , cnt , fail[MAXN] , sz[MAXN] , fk[MAXN] , f[MAXN];
int poi[MAXN];
void ins( char* O , int len , int o ) {
    int cur = 0;
    rep( i , 1 , len ) {
        if( !son[cur][O[i] - 'a'] ) son[cur][O[i] - 'a'] = ++ cnt , f[cnt] = cur;
        cur = son[cur][O[i] - 'a'];
    }
    poi[o] = cur;
}
void wkr( char* O ) {
    int cur = 0;
    rep( i , 1 , strlen( O + 1 ) ) {
        cur = son[cur][O[i] - 'a'];
        ++ sz[cur];
    }
}
vi G[MAXN];
void build( ) {
    queue<int> Q;
    for( int i = 0 ; i < 26 ; ++ i ) if( son[0][i] ) Q.push( son[0][i] );
    while( !Q.empty() ) {
        int u = Q.front(); Q.pop();
        for( int i = 0 ; i < 26 ; ++ i ) if( son[u][i] ) {
            int v = son[u][i];
            fail[v] = son[fail[u]][i];
            Q.push( v );
        } else son[u][i] = son[fail[u]][i];
    }
    rep( i , 1 , cnt ) G[fail[i]].pb( i );
}
```

## SAM

```
struct SAM {
    int son[MAXN][26] , par[MAXN] , len[MAXN] , sz[MAXN] , cnt , lst;
    SAM( ) : cnt(1) , lst(1) {  }
    void ins( char a ) {
```

```
        int x = a - 'a' , cur = ++ cnt , p = lst;
        len[cur] = len[lst] + 1 , sz[cur] = 1;
        while( p && !son[p][x] ) son[p][x] = cur , p = par[p];
        if( !p ) par[cur] = 1;
        else {
            int q = son[p][x];
            if( len[p] == len[q] + 1 ) par[cur] = q;
            else {
                int ct = ++ cnt;
                len[ct] = len[p] + 1 , par[ct] = par[q];
                memcpy( son[ct] , son[q] , sizeof son[q] );
                par[cur] = par[q] = ct;
                for( ; son[p][x] == q ; p = par[p] ) son[p][x] = ct;
            }
        }
        lst = cur;
    }
    vi G[MAXN];
    void ade( ) {
        rep( i , 2 , cnt ) G[par[i]].pb( i );
    }
    ll res = 0;
    void dfs( int u ) {
        for( int v : G[u] ) dfs( v ) , sz[u] += sz[v];
        if( sz[u] >= 2 ) res = max( res , sz[u] * 1ll * len[u] );
    }
} s ;
```

## SA

```
char ch[MAXN];
int sa[MAXN] , tp[MAXN] , rnk[MAXN] , buc[MAXN] , ht[MAXN] , mn[MAXN][20];
void init(  ) {
    n = strlen( ch + 1 ); int m = 130;
    for( int i = 1 ; i <= n ; ++ i ) ++ buc[rnk[i] = ch[i]];
    for( int i = 1 ; i <= m ; ++ i ) buc[i] += buc[i-1];
    for( int i = n ; i >= 1 ; -- i ) sa[buc[rnk[i]] --] = i;
    for( int k = 1 , p = 0 ; p < n ; k <<= 1 ) {
        p = 0;
        for( int i = 0 ; i <= m ; ++ i ) buc[i] = 0;
        for( int i = n - k + 1 ; i <= n ; ++ i ) tp[++p] = i;
        for( int i = 1 ; i <= n ; ++ i ) if( sa[i] > k ) tp[++p] = sa[i] - k;
        for( int i = 1 ; i <= n ; ++ i ) ++ buc[rnk[i]];
        for( int i = 1 ; i <= m ; ++ i ) buc[i] += buc[ i-1 ];
        for( int i = n ; i >= 1 ; -- i ) sa[buc[rnk[tp[i]]] --] = tp[i];
        p = 1;
        rep( i , 1 , n ) swap( rnk[i] , tp[i] );
        rnk[sa[1]] = 1;
        for( int i = 2 ; i <= n ; ++ i )
            rnk[sa[i]] = ( tp[sa[i]] == tp[sa[i-1]] && tp[sa[i] + k] == tp[sa[i-
1] + k] ) ? p : ++ p;
        m = p;
    }
    rep( i , 1 , n ) rnk[sa[i]] = i;
    for (int i = 1, k = 0; i <= n; ++i) {
```

```
        if (k) --k;
        while (ch[i + k] == ch[sa[rnk[i] - 1] + k]) ++k;
        ht[rnk[i]] = k;
    }
    rep( i , 1 , n ) mn[i][0] = ht[i];
    rep( k , 1 , 19 ) rep( i , 1 , n - ( 1 << k ) + 1 ) mn[i][k] = min( mn[i][k -
1] , mn[i + ( 1 << k - 1 )][k - 1] );
}

int gmn( int l , int r ) {
    int c = 31 - __builtin_clz( r - l + 1 );
    return min( mn[l][c] , mn[r - ( 1 << c ) + 1][c] );
}

int LCP( int a , int b ) {
    if( rnk[a] == rnk[b] ) return n - a + 1;
    if( rnk[a] > rnk[b] ) swap( a , b );
    return gmn( rnk[a] + 1 , rnk[b] );
}
```

## 后缀平衡树

后缀平衡树是一种维护后缀的数据结构，其中序遍历就是后缀数组。但是，对于它维护的后缀集合 $T$ 其中的一个字符串 $S \in T$，它可以支持 $O(\log n)$ 插入 $xS$。

我们考虑插入 $xS$ 的时候要做什么，首先会比较当前节点所代表的后缀的第一个字符和 $x$ 的大小关系，如果不相同，我们就已经成功比较了当前节点的后缀和这个串的大小关系。否则，我们需要比较 $S$ 与这个后缀去掉第一个字符（仍然是个后缀）的大小关系，也就是某两个后缀的大小关系。如果我们对后缀平衡树上每个点分配一个权值，可以 $O(1)$ 进行比较。分配权值可以给整个树分配一个 $[0, 10^9]$ 的权值，然后每次折半分配权值。使用替罪羊树来进行重构就可以让树高保持不高。

树上后缀排序同样可以用这个，每次插入的时候可以方便地找到去掉第一个字符的后缀 $fa[u]$。

```
char S[MAXN]; int fa[MAXN];

int siz[MAXN] , ch[MAXN][2]; double val[MAXN];
void pu( int rt ) {
    siz[rt] = siz[ch[rt][0]] + siz[ch[rt][1]] + 1;
}

int tmp[MAXN] , cn;
void pia( int x ) {
    if( !x ) return;
    pia( ch[x][0] ) , tmp[++ cn] = x , pia( ch[x][1] );
    ch[x][0] = ch[x][1] = 0;
}
void rebuild( int& rt , int l , int r , double L , double R ) {
    if( l > r ) return;
    int m = l + r >> 1; double M = ( L + R ) / 2;
    rt = tmp[m] , val[rt] = M;
    rebuild( ch[rt][0] , l , m - 1 , L , M ) , rebuild( ch[rt][1] , m + 1 , r , M
, R );
    pu( rt );
}
```

```
bool thisOneIsNotOK( int x ) {
    return siz[x] * alp < siz[ch[x][0]] || siz[x] * alp < siz[ch[x][1]];
}
void maintain( int& x , double L , double R ) {
    if( thisOneIsNotOK( x ) ) cn = 0 , pia( x ) , rebuild( x , 1 , cn , L , R );
}
bool cmp( int x , int w ) { // return if x is lower than w
    return ( S[x] != S[w] ? S[x] < S[w] : fa[x] == fa[w] ? x < w : val[fa[x]] <
val[fa[w]] );
}
void ins( int& x , int idx , double L , double R ) {
    if( !x ) {
        x = idx , val[idx] = ( L + R ) / 2 , siz[idx] = 1;
        return;
    }
    if( cmp( idx , x ) ) ins( ch[x][0] , idx , L , ( L + R ) / 2 );
    else ins( ch[x][1] , idx , ( L + R ) / 2 , R );
    pu( x ); maintain( x , L , R );
}
```

## Manacher

```
int n;
int len[MAXN << 1];
char s[MAXN << 1] , ch[MAXN];
void manacher( ) {
    rep( i , 1 , n ) {
        s[i << 1] = ch[i] , s[( i << 1 ) - 1] = '#';
    }
    s[0] = '~' , s[n = n << 1 | 1] = '#' , s[n + 1] = '@';
    int mx = 0 , ps = 0;
    rep( i , 1 , n ) {
        len[i] = i > mx ? 1 : min( len[( ps << 1 ) - i] , mx - i );
        while( s[i - len[i]] == s[i + len[i]] ) ++ len[i];
        if( i + len[i] > mx ) mx = i + len[i] , ps = i;
    }
}
void solve() {
    scanf("%s",ch + 1);
    n = strlen( ch + 1 );
    manacher();
    int mx = 0;
    rep( i , 1 , n ) mx = max( mx , len[i] - 1 );
    cout << mx << endl;
}
```

## PAM

```
int bc[MAXN];
struct PAM {
    int s[MAXN] , cn;
    int par[MAXN] , len[MAXN] , son[MAXN][26] , num[MAXN] , lst , cnt;
    void init( ) {
        cnt = 1;
```

```
        par[0] = 1 , len[0] = -1 , s[0] = -1;
    }
    int getfail( int x ) {
        while( s[cn] != s[cn - len[x] - 1] ) x = par[x];
        return x;
    }
    void add( int c ) {
        s[++ cn] = c;
        int p = getfail( lst );
        if( !son[p][c] ) {
            int q = ++ cnt;
            len[q] = len[p] + 2;
            int fl = getfail( par[p] );
            par[q] = !son[fl][c] ? 1 : son[fl][c];
            son[p][c] = q;
            num[q] = num[par[q]] + 1;
        }
        lst = son[p][c];
    }
} S ;

char ch[MAXN];
void solve() {
    S.init( );
    scanf("%s",ch + 1);
    n = strlen( ch + 1 );
    int las = 0;
    rep( i , 1 , n ) {
        ch[i] = ( ch[i] - 'a' + las ) % 26 + 'a';
        S.add( ch[i] - 'a' );
        printf("%d ",las = S.num[S.lst]);
    }
}
```

## 区间本质不同子串个数

因为是区间查询，所以我们可以把询问离线下来，然后依次加入字符并考虑$r$端点为这个字符的所有询问。然后我们需要实现两个操作：

- 插入一个字符
- 查询最晚的起始位置$\geq l$的本质不同字串个数

考虑插入一个字符会对答案造成的影响：显然会把$x$经过$x$的所有祖先最后到$root$的这条路径上的节点的最晚起始位置进行更新。这样就出现了一个问题：虽然LCT能进行链加/链覆盖，但是区间查询却做不到。那就考虑再维护一个线段树，线段树第$i$位代表$startpos = i$的本质不同子串的个数。

当先加入一个节点的时候，我们需要再线段树上移除会被更改的节点原来的贡献并加入新的贡献。而一个很显然的性质就是当$SAM$上一条从儿子到祖先路径代表的所有子串的最后一个$endpos$是相同的时，那么他们的$startpos$就是一段连续的区间。而根据LCT的性质，一个点到根的路径上只有$\log$个 `splay` ，所以我们可以根据某个节点属于哪个 `splay` 将路径上节点分为$\log$个类，每个类进行一次修改都能转化成线段树的区间加操作，然后就做完了。复杂度$O(m \log^2 n)$。

有个地方需要注意一下： 因为我们定义下的 `LCT` 是需要满足每个 `splay` 中的所有点的最大的 `endpos` 均相同，所以我们不能到处 `access` ，只有在修改的时候才能做一次 `access` 。

```cpp
#include "iostream"
#include "algorithm"
#include "cstring"
#include "cstdio"
#include "cmath"
#include "vector"
#include "map"
#include "set"
#include "queue"
using namespace std;
#define MAXN 200006
#define rep(i, a, b) for (int i = (a), i##end = (b); i <= i##end; ++i)
#define per(i, a, b) for (int i = (a), i##end = (b); i >= i##end; --i)
#define pii pair<int,int>
#define fi first
#define se second
#define mp make_pair
#define pb push_back
#define eb emplace_back
#define vi vector<int>
#define all(x) (x).begin() , (x).end()
#define mem( a ) memset( a , 0 , sizeof a )
typedef long long ll;
int n , m;
int A[MAXN];

struct SAM {
    int son[MAXN][26] , par[MAXN] , len[MAXN];
    int lst , cnt;
    void init( ) {
        lst = cnt = 1;
    }
    void ins( int x ) {
        int cur = ++ cnt;
        len[cur] = len[lst] + 1;
        int p = lst;
        while( p && !son[p][x] ) son[p][x] = cur , p = par[p];
        if( !p ) par[cur] = 1;
        else {
            int q = son[p][x];
            if( len[q] == len[p] + 1 ) par[cur] = q;
            else {
                int ct = ++ cnt;
                len[ct] = len[p] + 1 , par[ct] = par[q];
                memcpy( son[ct] , son[q] , sizeof son[q] );
                par[q] = par[cur] = ct;
                for( ; son[p][x] == q ; p = par[p] ) son[p][x] = ct;
            }
        }
        lst = cur;
    }
} S;

namespace SGT { // segment add , segment query
    long long T[MAXN << 2] , lz[MAXN << 2];
    inline void pu( int rt ) {
```

```
        T[rt] = T[rt << 1] + T[rt << 1 | 1];
    }
    inline void pd( int rt , int m ) {
        if( lz[rt] ) {
            lz[rt << 1] += lz[rt] , lz[rt << 1 | 1] += lz[rt];
            T[rt << 1] += ( m - ( m >> 1 ) ) * 1ll * lz[rt] , T[rt << 1 | 1] += (
m >> 1 ) * 1ll * lz[rt];
            lz[rt] = 0;
        }
    }
    void add( int rt , int l , int r , int L , int R , int c ) {
        if( L <= l && R >= r ) { T[rt] += 1ll * c * ( r - l + 1 ) , lz[rt] += c;
return; }
        pd( rt , r - l + 1 );
        int m = l + r >> 1;
        if( L <= m ) add( rt << 1 , l , m , L , R , c );
        if( R > m ) add( rt << 1 | 1 , m + 1 , r , L , R , c );
        pu( rt );
    }
    long long que( int rt , int l , int r , int L , int R ) {
        if( L <= l && R >= r ) return T[rt];
        pd( rt , r - l + 1 );
        int m = l + r >> 1; long long ret = 0;
        if( L <= m ) ret += que( rt << 1 , l , m , L , R );
        if( R > m ) ret += que( rt << 1 | 1 , m + 1 , r , L , R );
        return ret;
    }
}

namespace LCT {
    int ch[MAXN][2] , fa[MAXN] , lz[MAXN] , s[MAXN];
    inline bool notr( int u ) {
        return ch[fa[u]][0] == u || ch[fa[u]][1] == u;
    }
    inline void cover( int u , int c ) {
        s[u] = lz[u] = c;
    }
    inline void pd( int u ) {
        if( lz[u] ) {
            if( ch[u][0] ) cover( ch[u][0] , lz[u] );
            if( ch[u][1] ) cover( ch[u][1] , lz[u] );
            lz[u] = 0;
        }
    }
    inline void rotate( int x ) {
        int f = fa[x] , g = fa[f] , w = ch[fa[x]][1] == x;
        int wf = ch[g][1]==f , k = ch[x][w^1];
        if( notr(f) ) ch[g][wf] = x; ch[f][w] = k , ch[x][w^1] = f;
        fa[f] = x , fa[k] = f , fa[x] = g;
    }
    void upd( int u ) {
        if( notr( u ) ) upd( fa[u] );
        pd( u );
    }
    void splay( int u ) {
        upd( u );
```

```
        int f , g;
        while( notr( u ) ) {
            f = fa[u] , g = fa[f];
            if( notr( f ) ) rotate( ( ( ch[f][1] == u ) ^ ( ch[g][1] == f ) ) ? u
: f );
            rotate( u );
        }
    }
    void ac( int x , int r ) {
        using namespace SGT;
        int p , ls;
        for( p = 0 , ls ; x ; p = x , x = fa[x] ) {
            splay( x ) , ch[x][1] = p;
            if( ls = s[x] )
                add( 1 , 1 , n , ls - S.len[x] + 1 , ls - S.len[fa[x]] , -1 );
        }
        cover( p , r );
        add( 1 , 1 , n , 1 , r , 1 );
    }
}

char ch[MAXN];
int pos[MAXN];
long long ans[MAXN];
struct ques {
    int l , r , idx;
    void in( int i ) {
        idx = i;
        scanf("%d%d",&l,&r);
    }
} Q[MAXN];

bool cmp( ques a , ques b ) {
    return a.r == b.r ? a.l < b.l : a.r < b.r;
}

void solve() {
    scanf("%s",ch + 1);
    n = strlen( ch + 1 );
    S.init();
    rep( i , 1 , n ) S.ins( ch[i] - 'a' ) , pos[i] = S.lst;
    rep( i , 2 , S.cnt ) LCT::fa[i] = S.par[i];// , printf("%d %d\n",S.par[i],i);
    cin >> m;
    int l , r;
    rep( i , 1 , m )
        Q[i].in( i );
    sort( Q + 1 , Q + 1 + m , cmp );
    int cur = 1;
    rep( i , 1 , n ) {
        LCT::ac( pos[i] , i );
        while( cur <= m && Q[cur].r == i )
            ans[Q[cur].idx] = SGT::que( 1 , 1 , n , Q[cur].l , Q[cur].r ) , ++
cur;
    }
    rep( i , 1 , m ) printf("%lld\n",ans[i]);
}
```

```
signed main() {
//  freopen("input","r",stdin);
//  freopen("out","w",stdout);
//    int T;cin >> T;while( T-- ) solve();
    solve();
}
```

# 数论

## 杜教筛

设现在要求积性函数 $f$ 的前缀和， 设 $\sum_{i=1}^{n} f(i) = S(n)$。

再找一个积性函数 $g$， 则考虑它们的狄利克雷卷积的前缀和

$$\begin{aligned}
&\sum_{i=1}^{n}(f*g)(i)\\
&=\sum_{i=1}^{n}\sum_{d|i} f(d)g(\frac{i}{d})\\
&=\sum_{d=1}^{n} g(d)\sum_{i=1}^{\lfloor\frac{n}{d}\rfloor} f(i)\\
&=\sum_{d=1}^{n} g(d)S(\lfloor\frac{n}{d}\rfloor)
\end{aligned}$$

$$g(1)S(n) = \sum_{i=1}^{n}(f*g)(i) - \sum_{i=2}^{n} g(i)S(\lfloor\frac{n}{i}\rfloor)$$

$$\begin{aligned}
&1.\ \mu * I = \epsilon\\
&2.\ \varphi * I = id\\
&3.\ \mu * id = \varphi
\end{aligned}$$

```
#define MAXN 5000006
ll phi[MAXN] , mu[MAXN];
int tot , p[MAXN];
void init() {
    mu[1] = 1 , phi[1] = 1;
```

```
    for (int i = 2; i < MAXN; i++) {
    if (!p[i]) mu[i] = -1, p[++tot] = i , phi[i] = i - 1;
        for (int j = 1; j <= tot && i * p[j] < MAXN ; j++) {
        p[i * p[j]] = 1;
        if (i % p[j] == 0) { mu[i * p[j]] = 0 , phi[i*p[j]] = phi[i] * p[j];
break; }
        mu[i * p[j]] = -mu[i] , phi[i * p[j]] = phi[i] * (p[j] - 1);
    }
    }
    for( int i = 1 ; i < MAXN ; ++ i )
        phi[i] = phi[i-1] + phi[i] , mu[i] = mu[i-1] + mu[i];
}
unordered_map<int,ll> phii,muu;
ll getphi( int x ) {
    if( x < MAXN ) return phi[x];
    if( phii[x] ) return phii[x];
    ll ans = (ll)x * ( x + 1 ) / 2;
    for( int l = 2 , r ; l <= x ; l = r + 1 ) {
        r = x / ( x / l );
        ans -= (ll)( r - l + 1 ) * getphi( x / l );
    }
    return ( phii[x] = ans );
}
ll getmu( int x ) {
    if( x < MAXN ) return mu[x];
    if( muu[x] ) return muu[x];
    ll ans = 1;
    for( int l = 2 , r ; l <= x ; l = r + 1 ) {
        r = x / ( x / l );
        ans -= (ll)( r - l + 1 ) * getmu( x / l );
    }
    return ( muu[x] = ans );
}
```

# Min25 筛

## 预处理

设完全积性函数$F'(x)$，在质数处取值$F(p)=F'(p)$。

预处理一个$g$函数。$g(n,j)=\sum_{i=1}^{n}F'(i)[i\in\mathbb{P}\text{ or }\min_{p|i}p>P_j]$

也就是$i$是质数或者$i$最小值因子$>P_j$的贡献（注意求的是$F'$）

转移：

$$g(n,j)=\begin{cases}g(n,j-1)(P_j^2>n)\\g(n,j-1)-F'(P_j)[g(\frac{n}{P_j},j-1)-g(P_{j-1},j-1)](P_j\le n)\end{cases}$$

考虑$g(n,j-1)-g(n,j)$。如果$P_j^2>n$显然没有多余贡献。

否则会多余的一定是$P_j$的倍数，那么因为$F'$是完全积性函数，可以将多余的数全部除以$P_j$，并乘上贡献$F'(P_j)$。那么减掉的是$F'(P_j)$乘【$\frac{n}{P_j}$以内最小质因数$\ge P_j$的数的贡献和】。这个值就是$g(\frac{n}{P_j},j-1)-g(P_{j-1},j-1)$。前面多算了$<P_j$的质数，后面减掉了这一部分。

可以看出只有$\sqrt{n}$内的质数有用，线性筛到$\sqrt{n}$就行了。

后面的$g(P_{j-1},j-1)$按照定义等于$\sum_{i=1}^{P_{j-1}}F'(i)$。线性筛时预处理一下就行了。

这个$g$后面只用了质数处的$F'$，所以是对的~~而且很妙~~

## 求值

设$S(n,j)=\sum_{i=1}^{n}F(i)[\min_{p|i}p>P_j]$，即最小质因数大于$P_j$的所有数的$F$和，那么要求的答案是$S(n,0)$。

$S$的递推式为：$S(n,j)=g(n,|P|)-\sum_{i=1}^{j-1}f(P_i)+\sum_{p_k^e\le n,k>j}F(p_k^e)(S(\frac{n}{p_k^e},k)+[e>1])$

前面部分是质数的贡献，后面是合数的

后面的$\sum$意思就是枚举合数最小的质因子及次数，因为是积性函数可以直接乘起来。$[e>1]$意思是如果$e>1$那么有一个$p_k^e$没算进答案（$S$不计算1的贡献，但$e=1$时$p_k^e$就是质数已经在前面算过了），要算进答案。

```
namespace Min_25 {
    ll n; int B;
    int pri[MAXN] , en , sp[MAXN] , sp2[MAXN];
    void sieve( ) {
        B = sqrt( n + 0.5 );
        rep( i , 2 , B ) {
            if( !pri[i] ) pri[++ en] = i , sp[en] = ( sp[en - 1] + i ) % P ,
sp2[en] = ( sp2[en - 1] + i * 1ll * i ) % P;
            for( int j = 1 ; j <= en && i * pri[j] <= B ; ++ j ) {
                pri[i * pri[j]] = 1;
                if( i % pri[j] == 0 ) break;
            }
        }
    }
    int cc( ll x ) { x %= P; return ( x * ( x + 1 ) / 2 + P - 1 ) % P; }
```

```
    int cc2( ll x ) { x %= P; return ( x * ( x + 1 ) / 2 % P * ( 2 * x + 1 ) % P
* iv3 + P - 1 ) % P; }
    ll g1[MAXN] , g2[MAXN] , A[MAXN];
    int A1[MAXN] , A2[MAXN];
    int po( ll x ) {
        return x <= B ? A1[x] : A2[n / x];
    }
    void getG( ) {
        int tot = 0;
        for( ll l = 1 , r ; l <= n ; l = r + 1 ) {
            r = n / ( n / l );
            ++ tot;
            A[tot] = n / l;
            if( A[tot] <= B ) A1[A[tot]] = tot; else A2[n / A[tot]] = tot;
            g1[tot] = cc( A[tot] ) , g2[tot] = cc2( A[tot] );
        }
        rep( i , 1 , en )
            for( int j = 1 ; pri[i] * 1ll * pri[i] <= A[j] ; ++ j ) {
                g1[j] = ( g1[j] + P - pri[i] * 1ll * ( g1[po( A[j] / pri[i] )] +
P - sp[i - 1] ) % P ) % P;
                g2[j] = ( g2[j] + P - pri[i] * 1ll * pri[i] % P * ( g2[po( A[j] /
pri[i] )] + P - sp2[i - 1] ) % P ) % P;
            }
    }

    int g( int x ) {
        return ( g2[x] < g1[x] ? g2[x] - g1[x] + P : g2[x] - g1[x] );
    }
    int f( ll x ) {
        x %= P;
        return x * ( x - 1 ) % P;
    }

    int S( ll n , int k ) {
        if( n <= pri[k] ) return 0;
        int re = ( 1ll * g( po( n ) ) + P - sp2[k] + sp[k] ) % P;
        for( int i = k + 1 ; pri[i] * 1ll * pri[i] <= n && i <= en ; ++ i ) {
            ll cur = pri[i];
            for( int e = 1 ; cur <= n ; ++ e ) {
                re = ( re + f( cur ) * 1ll * ( S( n / cur , i ) + ( e != 1 ) ) )
% P;
                cur *= pri[i];
            }
        }
        return re;
    }

    int solve( ll x ) {
        n = x;
        sieve( );
        getG( );
        return ( S( n , 0 ) + 1 ) % P;
    }
}
```

# Powerful Number 筛

Powerful Number 筛也是一种筛，它是 $O(\sqrt{n})$ 的。应用时必须满足

- 它是积性函数
- 这个东西在质数处取值的函数可以快速计算前缀和。例如此题，显然有 $f(p)=1$ 。

首先 Powerful Number 的定义，是所有质因子次数都 $\geq 2$ 的数。这种数的个数是 $O(\sqrt{n})$ 级别的。因为显然，每个这样的数都可以被表示成 $a^2b^3$ ，所以寻找这种数的时候可以暴力枚举这里的 $a,b$ ，复杂度是 $\sum_{i=1}^{\sqrt{n}}(\frac{n}{i^2})^{\frac{1}{3}}$ ，这个东西积分一下会发现是 $O(\sqrt{n})$ 的。

现在，假设我们已经有了一个函数 $f'(p)=f(p),p\in \text{prime}$ 。现在我们想通过这个东西容斥出 $f$ 。具体来说，我们来尝试找到一个容斥系数 $h$ ，满足

$$f(x)=\sum_{d|x}f'(d)h(\frac{d}{x})$$
$$h=f/f'$$

显然我们有

$$h(1)=1$$
$$h(1)f'(p)+h(p)f'(1)=f(p)$$
$$f'(p)+h(p)f'(1)=f(p)$$

因此我们有 $h(p)=0$ 。所以 $h$ 仅在 Powerful Number 处有值！

因此有

$$\begin{aligned}&\sum_{i=1}^{n}f(i)\\=&\sum_{i=1}^{n}\sum_{pq=i}h(p)f'(q)\\=&\sum_{pq\leq n}h(p)f'(q)\\=&\sum_{p}h(p)\sum_{q\leq n/p}f'(q)\end{aligned}$$

后面那个东西的前缀和事可以求的，而前面这个东西仅在 Powerful Number 处取值。

```
int pri[MAXN] , en;
void sieve( ) {
    rep( i , 2 , 2e6 ) {
        if( !pri[i] ) pri[++ en] = i;
        for( int j = 1 ; j <= en && i * pri[j] <= 2000000 ; ++ j ) {
            pri[i * pri[j]] = 1;
            if( i % pri[j] == 0 ) {
                break;
            }
        }
    }
}

ll iv[133];

ll calc( ll x ) {
    return mul( mul( x , x + 1 ) , iv[2] );
}

ll dfs( ll k , ll lst , ll h ) {
    ll ans = mul( h , calc( n / lst ) );
    for( ll g = n / lst ; 1ll * pri[k] * pri[k] <= g ; ++ k ) {
        ll vh = P - mul( pri[k] * 1ll * pri[k] , iv[2] ) , hh = vh;
        for( __int128 e = 2 , p = pri[k] * 1ll * pri[k] ; p <= g ; p = p * pri[k]
, ++ e ) {
            ans = qm( ans + dfs( k + 1 , lst * p , mul( h , hh ) ) );
```

```
            vh = mul( vh , pri[k] );
            hh = ( mul( p * pri[k] , iv[e + 1] ) + P - p * pri[k] + P - vh ) % P;
            vh = ( vh + hh ) % P;
        }
    }
    return ans;
}
```

## CRT & exgcd & exCRT & 慢速乘

中国剩余定理 (Chinese Remainder Theorem, CRT) 可求解如下形式的一元线性同余方程组（其中 $n_1, n_2, \cdots, n_k$ 两两互质）：

$$\begin{cases} x & \equiv a_1 \pmod{n_1} \\ x & \equiv a_2 \pmod{n_2} \\ & \vdots \\ x & \equiv a_k \pmod{n_k} \end{cases}$$

上面的「物不知数」问题就是一元线性同余方程组的一个实例。

### 过程

1. 计算所有模数的积 $n$；
2. 对于第 $i$ 个方程：
    a. 计算 $m_i = \frac{n}{n_i}$；
    b. 计算 $m_i$ 在模 $n_i$ 意义下的 逆元 $m_i^{-1}$；
    c. 计算 $c_i = m_i m_i^{-1}$（**不要对 $n_i$ 取模**）。
3. 方程组在模 $n$ 意义下的唯一解为：$x = \sum_{i=1}^{k} a_i c_i \pmod{n}$。

```
LL CRT(int k, LL* a, LL* r) {
  LL n = 1, ans = 0;
  for (int i = 1; i <= k; i++) n = n * r[i];
  for (int i = 1; i <= k; i++) {
    LL m = n / r[i], b, y;
    exgcd(m, r[i], b, y);  // b * m mod r[i] = 1
    ans = (ans + a[i] * m * b % n) % n;
  }
  return (ans % n + n) % n;
}
```

```
ll mul( ll a , ll b , ll p ) { a %= p , b %= p; return ( (a * b - (ll)( (ll)(
(long double)a / p * b + 0.5 ) * p )) % p + p ) % p; }
int n;
ll A[MAXN] , B[MAXN];
ll gcd( int a , int b ) { return b ? a : gcd( b , a % b ); }
void exgcd( ll a , ll b , ll& d , ll& x , ll& y ) {
    if( !b ) { d = a , x = 1 , y = 0; return; }
    else exgcd( b , a % b , d , y , x ) , y -= x * ( a / b );
```

```
}
bool crt( ll& a1 , ll a2 , ll& b1 , ll b2 ) {
    ll d = a2 - a1;
    ll g , k1 , k2;
    exgcd( b1 , b2 , g , k1 , k2 );
    if( d % g ) return 0;
    else {
        ll r = b2 / g;
        k1 = mul( k1 , d / g , r );
        a1 = k1 * b1 + a1;
        b1 = ( b1 * r );
        return 1;
    }
}
ll excrt(  ) {
    ll a1 = A[0] , b1 = B[0] , a2 , b2;
    for( int i = 1 ; i < n ; ++ i ) {
        a2 = A[i] , b2 = B[i];
        if( !crt( a1 , a2 , b1 , b2 ) ) return -1;
    }
    return a1;
}
```

## 二次剩余

```
int n , P;
int i2;
struct comp {
    int a , b;
    comp( int a , int b ) : a(a) , b(b) {}
};
comp operator *( comp a , comp b ) { return comp( ( 1ll * a.a * b.a % P + 1ll *
a.b * b.b % P * i2 % P ) % P , ( 1ll * a.a * b.b % P + 1ll * a.b * b.a % P ) % P
); }
comp Pow( comp a , int x ) {
    comp ans( 1 , 0 );
    while( x ) {
        if( x & 1 ) ans = ans * a;
        a = a * a , x >>= 1;
    }
    return ans;
}
int work( ) {
    if( n == 0 ) return 0;
    if( P == 2 && n == 1 ) return 1;
    if( n >= P || Pow( comp(n , 0) , (P - 1) / 2 ).a == P - 1 ) return -1;
    int t;
    for( t = rand() % P + 1 ; ; t = rand() % P + 1 ) {
        i2 = ( 1ll * t * t % P - n + P) % P;
        if( Pow( comp( i2 , 0 ) , ( P - 1 ) / 2 ).a != P - 1 ) continue;
        break;
    }
    comp ans = Pow( comp( t , 1 ) , ( P + 1 ) / 2 );
    return min( ans.a , P - ans.a );
```

```
}
```

## 逆元 (exgcd & 递推)

```
void exgcd(ll a,ll b,ll& d,ll& x,ll& y){
    if(!b) x=1,y=0,d=a;
    else exgcd(b,a%b,d,y,x),y-=x*(a/b);
}
ll inv(ll a,ll p){
    ll d,x,y;
    exgcd(a,p,d,x,y);
    return d==1?(x+p)%p:-1;
}

for( int i = 2 ; i <= n ; ++ i )
        inv[i] = 1ll * ( p - p / i ) * inv[p % i] % p , printf("%lld\n",inv[i]);
```

## Pollard-Rho

```
ll mul( ll x , ll y , ll p ) {
    return ( x * y - (ll) ( (long double) x / p * y + 0.5 ) * p + p ) % p;
}
ll Pow( ll x , ll a , ll p ) {
    ll ret = 1;
    while( a ) {
        if( a & 1 ) ret = mul( ret , x , p );
        x = mul( x , x , p ) , a >>= 1;
    }
    return ret;
}

const int _p[] = { 2,3,5,7,11,13,17,19,23 } , _l = 9;
bool millerRabin( ll x ) {
    if( x == 1 ) return false;
    ll t = x - 1;
    int cnt = 0;
    while( ~t & 1 ) t >>= 1 , ++ cnt;
    rep( i , 0 , _l - 1 ) {
        if( x == _p[i] ) return true;
        ll a = Pow( _p[i] , t , x ) , nex;
        rep( i , 1 , cnt ) {
            nex = mul( a , a , x );
            if( nex == 1 && a != 1 && a != x - 1 ) return false;
            a = nex;
        }
        if( a != 1 ) return false;
    }
    return true;
}

ll f( ll x , ll c , ll p ) {
    return ( mul( x , x , p ) + c ) % p;
}
```

```
ll _rand( ) {
    return (long long)rand() << 32 | rand();
}

ll gcd( ll a , ll b ) {
    return !b ? a : gcd( b , a % b );
}

ll pollardRho( ll x ) {
    ll s = 0 , t = 0 , c = _rand() % ( x - 1 ) + 1 , val = 1 , g;
    for( int cyc = 1 ; ; cyc <<= 1 , s = t ) {
        for( int i = 0 ; i < cyc ; ++ i ) {
            t = f( t , c , x ) , val = mul( val , abs( t - s ) , x );
            if( i % 127 == 0 ) {
                g = gcd( val , x );
                if( g != 1 ) return g;
            }
        }
        g = gcd( val , x );
        if( g != 1 ) return g;
    }
}

vector<ll> divs;
void analyze( ll x ) {
    if( x == 1 ) return;
    if( millerRabin( x ) ) { divs.pb( x ); return; }
    ll d = x;
    while( d == x ) d = pollardRho( x );
    x /= d;
    analyze( d ) , analyze( x );
}

void solve() {
    ll n;
    scanf("%lld",&n);
    divs.clear();
    analyze( n );
    sort( all( divs ) );
    if( divs.size() == 1 ) puts("Prime");
    else printf("%lld\n",divs.back());
}
```

## 扩展欧拉定理

$$b \geq \varphi(m)\text{时}, a^b \equiv a^{(b \mod \varphi(m))+\varphi(m)} \mod m$$

## 原根

```
vi divs( int x ) {
    int B = sqrt( x ) + 1;
    vi as;
    rep( i , 2 , B ) if( x % i == 0 ) {
        as.pb( i );
        while( x % i == 0 ) x /= i;
    }
    if( x != 1 ) as.pb( x );
    return as;
}
vi we = divs( P - 1 );
int wn = 0;
rep( i , 2 , P - 1 ) {
    int flg = 0;
    for( int x : we ) if( Pow( i , ( P - 1 ) / x ) == 1 ) {
        flg = 1; break;
    }
    if( !flg ) {
        wn = i; break;
    }
}
```

## BSGS & exBSGS

```
int gcd( int a , int b ) { return !b ? a : gcd( b , a % b ); }

int Pow( int x , int a ) {
    int ret = 1;
    while( a ) {
        if( a & 1 ) ret = ret * 1ll * x % P;
        x = x * 1ll * x % P, a >>= 1;
    }
    return ret;
}

unordered_map<int,int> M;
int bsgs( int a , int b , int p , int w = 1 ) {
    a %= p , b %= p , w %= p;
    if( w == b ) return 0;
    int blo = ceil( sqrt( p ) ) , ls = 1;
    for( int i = 0 ; i < blo ; ++ i , ls = ls * 1ll * a % p )
        M[ls] = i;
    w = Pow( b , P - 2 );
    for( int i = 1 , nw = w * 1ll * ls % p ; i <= blo ; ++ i , nw = nw * 1ll * ls
% p )
        if( M.count( nw ) ) return i * blo - M[nw];
    return -1;
}

int exbsgs( int a , int b , int p ) {
    if( b == 1 ) return 0;
    M.clear();
```

```
    a %= p , b %= p;
    int w = 1 , g , ls = 0;
    while( ( g = gcd( a , p ) ) != 1 ) {
        if( b % g ) return -1;
        w = w * 1ll * ( a / g ) % p , ++ ls;
        b /= g , p /= g;
    }
    int res = bsgs( a , b , p , w );
    return ( ~res ? res + ls : -1 );
}
```

## exLucas

$a, b \leq 10^{18}, p \leq 10^6$

```
int Pow( int x , int a , int p ) {
    int ret = 1;
    while( a ) {
        if( a & 1 ) ret = ret * 1ll * x % p;
        x = x * 1ll * x % p , a >>= 1;
    }
    return ret;
}

void exgcd( int a , int b , int& d , int& x , int& y ) {
    if( !b ) d = a , x = 1 , y = 0;
    else exgcd( b , a % b , d , y , x ) , y -= a / b * x;
}

int inv( int a , int p ) {
    int d , x , y;
    exgcd( a , p , d , x , y );
    return d == 1 ? ( x + p ) % p : -1;
}

int f( ll n , int p , int pk ) {
    if( n == 0 ) return 1;
    int re = 1;
    rep( i , 1 , pk ) if( i % p ) re = re * 1ll * i % pk;
    re = Pow( re , n / pk % ( pk - 1 ) , pk );
    for( ll i = n - n % pk + 1 ; i <= n ; ++ i ) if( i % p ) re = re * 1ll * ( i
% pk ) % pk;
    return re * 1ll * f( n / p , p , pk ) % pk;
}

int g( ll n , int p ) {
    if( n < p ) return 0;
    return g( n / p , p ) + n / p;
}

int C( ll a , ll b , int p , int pk ) { // C a b % pk
    int re = f( a , p , pk ) * 1ll * inv( f( b , p , pk ) , pk ) % pk * inv( f( a
- b , p , pk ) , pk ) % pk;
    int t = Pow( p , g( a , p ) - g( b , p ) - g( a - b , p ) , pk );
    return 1ll * re * t % pk;
```

```
}

int re[MAXN] , d[MAXN] , en;

void solve() {
    ll n , m; int p;
    cin >> n >> m >> p;
    int c = p;
    for( int i = 2 ; i * i <= c ; ++ i ) if( c % i == 0 ) {
        int k = 1;
        while( c % i == 0 ) c /= i , k *= i;
        d[++ en] = k , re[en] = C( n , m , i , k );
    }
    if( c != 1 ) d[++ en] = c , re[en] = C( n , m , c , c );
    int ans = 0;
    rep( i , 1 , en ) {
        int M = p / d[i] , t = inv( M , d[i] );
        ans = ( ans + 1ll * re[i] * M % p * t % p ) % p;
    }
    printf("%d\n",ans);
}
```

# 数据结构

## splay

```
int RT , siz[MAXN] , val[MAXN] , cnt[MAXN] , idx , ch[MAXN][2] , fa[MAXN];

inline void pu( int rt ) {
    siz[rt] = siz[ch[rt][0]] + cnt[rt] + siz[ch[rt][1]];
}

inline int newnode( int x , int f ) {
    ++idx , cnt[idx] = siz[idx] = 1 , val[idx] = x , ch[idx][0] = ch[idx][1] = 0
, fa[idx] = f;
    return idx;
}

void rotate( int u ) {
    int f = fa[u] , g = fa[f] , w = ch[f][1] == u , k = ch[u][w ^ 1];
    if( g ) ch[g][ch[g][1] == f] = u;
    ch[u][w ^ 1] = f , ch[f][w] = k;
    fa[f] = u , fa[k] = f , fa[u] = g;
    pu( f ) , pu( u );
}

void splay( int u , int to = 0 ) {
    if (!to) RT = u;
    int f , g;
    while( fa[u] != to ) {
        f = fa[u] , g = fa[f];
        if( g != to ) rotate( ( ch[f][1] == u ^ ch[g][1] == f ) ? u : f );
        rotate( u );
    }
```

```
}

int poi;

void ins( int &u , int x , int f ) {
    if( !u ) {
        poi = u = newnode( x , f );
        return;
    }
    if( val[u] == x ) {
        ++cnt[u];
        pu( u ) , poi = u;
        return;
    }
    ins( x < val[u] ? ch[u][0] : ch[u][1] , x , u );
    pu( u );
}

int rnk( int u , int x ) {
    if( !u ) return 0;
    if( val[u] == x ) return poi = u , siz[ch[u][0]] + 1;
    return poi = u , x < val[u] ? rnk( ch[u][0] , x ) : rnk( ch[u][1] , x ) +
siz[ch[u][0]] + cnt[u];
}

int find( int u , int x ) {
    if( !u ) return 0;
    if( x <= siz[ch[u][0]] ) return poi = u , find( ch[u][0] , x );
    if( x <= siz[ch[u][0]] + cnt[u] ) return poi = u , val[u];
    return poi = u , find( ch[u][1] , x - siz[ch[u][0]] - cnt[u] );
}

int mx;

void Pre( int u , int x ) {
    if( !u ) return;
    if( val[u] < x && val[u] > mx ) mx = val[u] , poi = u;
    Pre( x <= val[u] ? ch[u][0] : ch[u][1] , x );
}

int pre( int u , int x ) {
    mx = -2e9 - 6 , poi = 0;
    Pre( u , x );
    return mx;
}

void Nxt( int u , int x ) {
    if( !u ) return;
    if( val[u] > x && val[u] < mx ) mx = val[u] , poi = u;
    Nxt( x >= val[u] ? ch[u][1] : ch[u][0] , x );
}

int nxt( int u , int x ) {
    mx = 2e9 + 6 , poi = 0;
    Nxt( u , x );
    return mx;
```

```
}

void era( int u , int x ) {
    pre( u , x );
    assert( poi );
    splay( poi );
    int rt = poi;
    nxt( rt , x );
    assert( poi );
    splay( poi , rt );
    int ps = ch[ch[rt][1]][0];
    if( !ps ) return;
    if( cnt[ps] > 1 ) -- cnt[ps] , pu( ps );
    else ch[ch[rt][1]][0] = 0;
    pu( ch[rt][1] ) , pu( rt );
}

void solve( ) {
    cin >> n;
    int op , x;
    ins( RT , -2e9 , 0 );
    ins( RT , 2e9 , 0 );
    cnt[1] = siz[1] = cnt[2] = siz[2] = 0;
    rep( i , 1 , n ) {
        scanf("%d%d",&op,&x);
        if( op == 1 ) ins( RT , x , 0 ) , splay( poi );
        else if( op == 2 ) era( RT , x );
        else if( op == 3 ) printf("%d\n",rnk( RT , x ) );
        else if( op == 4 ) printf("%d\n",find( RT , x ) );
        else if( op == 5 ) printf("%d\n",pre( RT , x ) );
        else printf("%d\n",nxt( RT , x ) );
        if( op != 2 ) splay( poi );
        poi = 0;
    }
}
```

## LCT

```
int fa[MAXN] , ch[MAXN][2] , rev[MAXN] , T[MAXN] , val[MAXN];
inline bool notroot( int x ) {
    return ch[fa[x]][0] == x || ch[fa[x]][1] == x;
}
inline void pushup( int rt ) {
    T[rt] = ( T[ch[rt][0]] ^ T[ch[rt][1]] ^ val[rt] );
}
void revers( int rt ) {
    swap( ch[rt][0] , ch[rt][1] ) , ( rev[rt] ^= 1 );
}
void pushdown( int rt ) {
    if( rev[rt] ) {
        if( ch[rt][0] ) revers( ch[rt][0] );
        if( ch[rt][1] ) revers( ch[rt][1] );
        rev[rt] = 0;
    }
}
```

```
void rotate( int x ) {
    int f = fa[x] , g = fa[f] , w = ch[fa[x]][1] == x;
    int wf = ch[g][1]==f , k = ch[x][w^1];
    if( notroot(f) ) ch[g][wf] = x; ch[f][w] = k , ch[x][w^1] = f;
    fa[f] = x , fa[k] = f , fa[x] = g;
    pushup( f ) , pushup( x );
}
void update( int x ) {
    if( notroot( x ) ) update( fa[x] );
    pushdown( x );
}
void splay( int x ) {
    update( x );
    int f , g;
    while( notroot( x ) ) {
        f = fa[x] , g = fa[f];
        if( notroot( f ) )
            rotate( (ch[f][0]==x)^(ch[g][0]==f) ? x : f );
        rotate( x );
    }
}
void access( int x ) {
    for( int p = 0 ; x ; ( p = x , x = fa[x] ) )
        splay( x ) , ch[x][1] = p , pushup( x );
}
void makeroot( int x ) {
    access( x ) , splay( x );
    revers( x );
}
int findroot( int x ) {
    access( x ) , splay( x );
    while( ch[x][0] ) pushdown( x ) , x = ch[x][0];
    splay( x );
    return x;
}
void link( int u , int v ) {
    makeroot( u );
    if( findroot( v ) == u ) return;
    fa[u] = v;
}
void cut( int u , int v ) {
    makeroot( u );
    if( findroot(v) == u && fa[v] == u && !ch[v][0] ) {
        fa[v] = ch[u][1] = 0;
        pushup( u );
    }
}
void split( int u , int v ) {
    makeroot( u );
    access( v );

    splay( v );
}

int main() {
    cin >> n >> m;
```

```
    for( int i = 1 ; i <= n ; ++ i ) scanf("%d",&val[i]);
    int u , v , opt;
    while( m-- ) {
        scanf("%d%d%d",&opt,&u,&v);
        if( opt == 0 ) {
            split( u , v ); printf("%d\n",T[v]);
        } else if( opt == 1 ) {
            link( u , v );
        } else if( opt == 2 ) {
            cut( u , v );
        } else {
            splay( u ); val[u] = v;
        }
    }
}
```

## 线段树合并

```
int rt[MAXN] , ls[MAXN * 42] , rs[MAXN * 42] , cnt;
pii T[MAXN * 42];
void pu( int rt ) {
    T[rt] = mp( -0x3f3f3f3f , 0 );
    if( ls[rt] ) T[rt] = T[ls[rt]];
    if( rs[rt] ) T[rt] = max( T[rt] , T[rs[rt]] );
}
int merge( int r1 , int r2 , int l , int r ) {
    if( !r1 || !r2 ) return r1 | r2;
    ++ cnt;
    int rt = cnt;
    if( l == r ) {
        T[rt] = mp( T[r1].fi + T[r2].fi , -l );
        return rt;
    }
    int m = l + r >> 1;
    ls[rt] = merge( ls[r1] , ls[r2] , l , m );
    rs[rt] = merge( rs[r1] , rs[r2] , m + 1 , r );
    pu( rt );
    return rt;
}
void add( int& rt , int l , int r , int p , int c ) {
    if( !rt ) rt = ++ cnt;
    if( l == r ) { T[rt].fi += c , T[rt].se = -l; return; }
    int m = l + r >> 1;
    if( p <= m ) add( ls[rt] , l , m , p , c );
    else add( rs[rt] , m + 1 , r , p , c );
    pu( rt );
}
```

## 李超树

```
struct tcurts {
    ll k , b;
    ll re( int x ) const {
        return k * x + b;
```

```
    }
} F[MAXN] ;

inline bool wanbao(const tcurts &A, const tcurts &B , int x)  { // A bei B zai x wanbao
    return A.re( x ) > B.re( x );
}

int T[MAXN << 2];
void ins( int rt , int l , int r , int d ) {
    if( !T[rt] ) { T[rt] = d; return; }
    int m = l + r >> 1;
    if( wanbao( F[T[rt]] , F[d] , m ) ) swap( T[rt] , d );
    if( l == r ) return;
    if( wanbao( F[d] , F[T[rt]] , l ) && wanbao( F[d] , F[T[rt]] , r ) ) return;
    if( wanbao( F[T[rt]] , F[d] , l ) ) ins( rt << 1 , l , m , d );
    else ins( rt << 1 | 1 , m + 1 , r , d );
}
ll que( int rt , int l , int r , int p ) {
    if( !T[rt] ) return 0x3f3f3f3f3f3f3f3f;
    int m = l + r >> 1;
    ll re = F[T[rt]].re( p );
    if( p <= m ) return min( re , que( rt << 1 , l , m , p ) );
    else return min( re , que( rt << 1 | 1 , m + 1 , r , p ) );
}
```

## 点分治

```
vi G[MAXN];

int vis[MAXN] , siz[MAXN] , mx[MAXN] , rt , pr;
void afs( int u , int fa ) {
    siz[u] = 1 , mx[u] = 0;
    for( int v : G[u] ) if( !vis[v] && v != fa ) {
        afs( v , u );
        siz[u] += siz[v] , mx[u] = max( mx[u] , siz[v] );
    }
}
void bfs( int u , int fa ) {
    mx[u] = max( mx[u] , siz[rt] - siz[u] );
    if( !pr || mx[u] < mx[pr] ) pr = u;
    for( int v : G[u] ) if( !vis[v] && v != fa )
        bfs( v , u );
}
inline int getit( int u ) {
    rt = u , pr = 0 , afs( rt , rt );
    bfs( rt , rt );
    return pr;
}

int fa[MAXN];
void divide( int u , int sz ) {
    vis[u] = 1;
    T1[u].T.resize( sz + 5 ) , T2[u].T.resize( sz + 5 );
    for( int v : G[u] ) if( !vis[v] ) {
```

```
        int r = getit( v );
        fa[r] = u;
        divide( r , siz[v] );
    }
}
```

## 边分治

```
int head[MAXN << 1], nex[MAXN << 2], to[MAXN << 2], wto[MAXN << 2], ecn = -1;

void ade(int u, int v, int w) {
//    cout << u << ' ' << v << ' ' << w << endl;
    to[++ecn] = v, nex[ecn] = head[u], head[u] = ecn, wto[ecn] = w;
    to[++ecn] = u, nex[ecn] = head[v], head[v] = ecn, wto[ecn] = w;
}

vector<pii> G[MAXN];

void rebuild( int u , int fa ) {
    int flg = 0 , cur = 0 , last = u;
    for( auto& t : G[u] ) {
        int v = t.fi;
        if( v == fa ) continue;
        if( !flg ) { flg = 1; ade( u , v , t.se ); }
        else {
            cur = ++ n;
            ade( last , cur , 0 ) , ade( cur , v , t.se );
            last = cur;
        }
        rebuild( v , u );
    }
}

int vis[MAXN << 1] , siz[MAXN << 1] , E , rt , mn;
void getsize( int u , int fa ) {
    siz[u] = 1;
    for( int i = head[u] ; ~i ; i = nex[i] ) {
        int v = to[i];
        if( v == fa || vis[i >> 1] ) continue;
        getsize( v , u );
        siz[u] += siz[v];
    }
}
void getedge( int u , int fa ) {
    for( int i = head[u] ; ~i ; i = nex[i] ) {
        int v = to[i];
        if( v == fa || vis[i >> 1] ) continue;
        if( max( siz[v] , siz[rt] - siz[v] ) < mn ) mn = max( siz[v] , siz[rt] -
siz[v] ) , E = ( i >> 1 );
        getedge( v , u );
    }
}
int getit( int u ) { // the edge (ret << 1) and (ret << 1 | 1) are the answer.
    rt = u , mn = 0x3f3f3f3f , E = -1; getsize( u , u ); getedge( u , u ); return
E;
```

```
}

int que[MAXN] , ans[MAXN];

int dis[MAXN] , buc[10000006];
vi pts;
void work1( int u , int fa ) {
    if( dis[u] <= 10000000 ) buc[dis[u]] = 1 , pts.pb( u );
    for( int i = head[u] ; ~i ; i = nex[i] ) {
        int v = to[i];
        if( v == fa || vis[i >> 1] ) continue;
        dis[v] = dis[u] + wto[i];
        work1( v , u );
    }
}
void work2( int u , int fa ) {
    rep( i , 1 , m ) ans[i] += dis[u] <= que[i] ? buc[que[i] - dis[u]] : 0;
    for( int i = head[u] ; ~i ; i = nex[i] ) {
        int v = to[i];
        if( v == fa || vis[i >> 1] ) continue;
        dis[v] = dis[u] + wto[i];
        work2( v , u );
    }
}

void solve( int e ) {
    vis[e] = 1;
    int u = to[e << 1] , v = to[e << 1 | 1];
    pts.clear();
    dis[u] = wto[e << 1] , work1( u , u );
    dis[v] = 0; work2( v , v );
    for( int x : pts ) buc[dis[x]] = 0;
    u = getit( u ) , v = getit( v );
    if( ~u ) solve( u );
    if( ~v ) solve( v );
}
```

## 二次离线莫队

查询区间逆序对数

首先我们显然会一个非常辣鸡的 $O(n\sqrt{m}\log n)$ 做法，就是莫队维护 BIT ，可惜看不出怎么优化。。

于是考虑上二次离线莫队。

设 $[a,b][c,d]$ 表示满足 $x\in[a,b],y\in[c,d],a_x>a_y$ 的数量，也就是两个区间之间的逆序对数。如果 $c<b$ 我们认为它没有定义。

我们考虑移动一个区间，比如从 $l,r$ 移动到 $l,r'$ ，增加量是：

$$\sum_{i=r+1}^{r'}[l,i-1][i,i]$$

显然后面那个符号可以拆成前缀的差：

$$\sum_{i=r+1}^{r'}[1,i-1][i,i]-[1,l-1][i,i]$$

我们考虑 $[1,i-1][i,i]$ 这个东西可以方便地预处理前缀和。于是考虑我们现在要算的是：

$$\sum_{i=r+1}^{r'}[1,l-1][i,i]$$

也就是：

$$[1,l-1][r+1,r']$$

这个东西怎么算呢?

我们把莫队需要的这种样子的询问都拿出来，把所有 $[r+1,r']$ 挂在 $l-1$ 下面就好了。一共会有 $m$ 个长成这样的询问，所以空间复杂度为 $O(m)$。并且由于莫队我们知道总的移动次数是不超过 $O(n\sqrt{m})$ 的，于是就可以从再次离线算这个东西，从左到右加入数，然后计算比 $[r+1,r']$ 内的数大的数字个数。为了让后面的暴力计算做到 $O(1)$ 我们可以根号平衡，值域分块一下，插入 $O(\sqrt{n})$ 查询 $O(1)$ ，插入的次数是不超过 $O(n)$ 的于是总复杂度就优化到了 $O(n\sqrt{m}+n\sqrt{n})$ 。（大概是这个复杂度吧，，不知道实际上有没有问题）

```
struct que {
    int l , r , id;
    inline void rd( int x ) {
        id = x; scanf("%d%d",&l,&r);
    }
} Q[MAXN] ;

long long ans[MAXN];
long long Ls[MAXN] , Rs[MAXN]; // sum of [1,i-1][i,i] or [i,i][i+1,n]

namespace fwk {
    int T[MAXN];
    void add( int x , int c ) {
        while( x <= n ) T[x] += c , x += ( x & -x );
    }
    int sum( int x ) {
        int ret = 0;
        while( x > 0 ) ret += T[x] , x -= ( x & -x );
        return ret;
    }
}
```

```
int BLO;
namespace kuai1 {
    int T[MAXN] , lz[400];
    void add( int x ) { // ( 1 ~ x - 1 ) + 1
        per( i , x , ( x - 1 ) / BLO * BLO + 1 ) ++ T[i];
        per( i , ( x - 1 ) / BLO , 1 ) ++ lz[i];
    }
    int get( int x ) {
        return T[x] + lz[( x - 1 ) / BLO + 1];
    }
}

namespace kuai2 {
    int T[MAXN] , lz[400] , tot;
    void add( int x ) { // ( 1 ~ x - 1 ) + 1
        rep( i , x , ( x - 1 ) / BLO * BLO + BLO ) ++ T[i];
        rep( i , ( x - 1 ) / BLO + 2 , tot ) ++ lz[i];
    }
    int get( int x ) {
        return T[x] + lz[( x - 1 ) / BLO + 1];
    }
}

struct dq { int l , r , id , c; };
vector<dq> Rqs[MAXN] , Lqs[MAXN]; // Rqs : [1,i][l,r] , Lqs : [l,r][i,n]

bool cmp( que a , que b ) {
    return bl[a.l] == bl[b.l] ? bl[a.l] & 1 ? a.r < b.r : a.r > b.r : a.l < b.l;
}

void solve() {
    cin >> n >> m;
    blo = n / sqrt( m * 2 / 3 ); BLO = sqrt( n );
    rep( i , 1 , n ) scanf("%d",A + i) , bl[i] = ( i - 1 ) / blo + 1 , a[i] =
A[i];
    sort( a + 1 , a + 1 + n );
    int sz = unique( a + 1 , a + 1 + n ) - a - 1;
    rep( i , 1 , n ) A[i] = lower_bound( a + 1 , a + 1 + sz , A[i] ) - a;
    rep( i , 1 , m ) Q[i].rd( i );
    sort( Q + 1 , Q + 1 + m , cmp );
    rep( i , 1 , n ) Ls[i] = fwk::sum( n - A[i] ) , fwk::add( n - A[i] + 1 , 1 );
    rep( i , 1 , n ) Ls[i] += Ls[i - 1];
    mem( fwk::T );
    per( i , n , 1 ) Rs[i] = fwk::sum( A[i] - 1 ) , fwk::add( A[i] , 1 );
    per( i , n , 1 ) Rs[i] += Rs[i + 1];
    int l = 1 , r = 0 , L , R , idx;
    rep( i , 1 , m ) {
        L = Q[i].l , R = Q[i].r , idx = Q[i].id;
        if( r < R ) Lqs[l - 1].eb( (dq){ r + 1 , R , idx , -1 } ) , ans[idx] +=
Ls[R] - Ls[r];
        if( r > R ) Lqs[l - 1].eb( (dq){ R + 1 , r , idx , 1 } ) , ans[idx] -=
Ls[r] - Ls[R];
        r = R;
        if( l > L ) Rqs[R + 1].eb( (dq){ L , l - 1 , idx , -1 } ) , ans[idx] +=
Rs[L] - Rs[l];
```

```
        if( l < L ) Rqs[R + 1].eb( (dq){ l , L - 1 , idx , 1 } ) , ans[idx] -=
Rs[l] - Rs[L];
        l = L;
    }
    long long re;
    rep( i , 1 , n ) {
        kuai1::add( A[i] - 1 );
        for( auto& t : Lqs[i] ) {
            re = 0;
            rep( j , t.l , t.r )
                re += kuai1::get( A[j] );
            ans[t.id] += t.c * re;
        }
    }
    kuai2::tot = ( sz - 1 ) / BLO + 1;
    per( i , n , 1 ) {
        kuai2::add( A[i] + 1 );
        for( auto& t : Rqs[i] ) {
            re = 0;
            rep( j , t.l , t.r )
                re += kuai2::get( A[j] );
            ans[t.id] += t.c * re;
        }
    }
    rep( i , 1 , m ) ans[Q[i].id] += ans[Q[i - 1].id];
    rep( i , 1 , m ) printf("%lld\n",ans[i]);
}
```

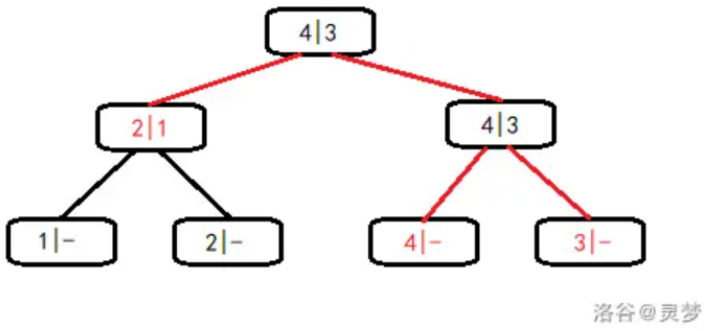
# jls 线段树

首先，让我们考虑只有操作 $2,3,4$ 的情况。我们发现，关于区间最大值的询问是可以使用带懒标记的线段树维护的，而区间和则不是很容易求出，因为这不能在下传标记时快速更新答案。

考虑到区间取 $\min$ 的操作只会对最大值不超过 $k$ 的节点产生影响，我们可以在这方面产生思路。为了使复杂度变对，线段树的一个节点要维护三个信息：区间最大值 $mx$，区间严格次大值 $se$ 和最大值的个数 $cnt$。那么，一次区间最值操作作用在这个节点上时，可以被分为以下三种情况：

- $k \geq mx$，此时该操作不会对当前节点产生影响，直接退出；
- $se < k < mx$，此时这个节点维护的区间中所有最大值都会被修改为 $k$，而最大值个数不变。将区间和加上 $cnt \cdot (k - mx)$，打上懒标记，然后退出即可；
- $k \leq se$，此时无法快速更新区间信息，因此我们需要继续递归到左右子树中，回溯时合并信息。

举个例子，现在要以 $k = 2$ 对下面这棵线段树维护的区间取 $\min$。每个节点左侧表示区间最大值，右侧表示严格次大值。（很抱歉用了吉老师论文里的例子，因为我实在找不出比它更适合做例子的例子了）




按照上面描述的操作，我们应该沿着红色的边 DFS，最后在红色的节点上更新区间和并打上标记。

这样做的复杂度可以证明是 $O(m \log n)$ 的。原论文的证明使用了势能分析，这里给出一种更好懂的 $O((n+m) \log n)$ 下界的证明：

> 显然只有暴力 DFS 的过程会影响复杂度，我们考虑一次区间最值操作中哪些节点会被搜索到。发现到达的节点区间中不同的数的个数（下称“值域”）一定会减少（因为至少将最大值与次大值合并了）。线段树每层节点表示的区间的值域一共是 $O(n)$ 的，一共 $\log$ 层加起来就是 $O(n \log n)$。再加上每次操作的复杂度，可以得到复杂度的一个下界为 $O((n+m) \log n)$。

然后，我们加入区间加减操作（操作 1）。发现上面方法的本质是把线段树上每个节点维护的元素分成了**最大值和非最大值**这两类，并对它们分别维护。区间最大值标记只会打在 $se < v < mx$ 的节点上，可以看成只针对最大值的加减操作；而区间加操作对于这两类值都生效。分别维护区间最大值的加减标记和非最大值的加减标记，下传时判断当前节点是否为区间最大值即可。

由于区间加减操作，某些节点的值域会增大。一次区间加减最多会影响 $O(\log^2 n)$ 个节点的值域，所以用之前的证明方法可证复杂度的一个上界为 $O(n \log n + m \log^2 n)$（论文里给的是 $O(m \log^2 n)$）。而实际实现时会发现这个上界其实往往是跑不满的，速度几乎与大常数一个 $\log$ 接近。

目前为止，我们已经解决了测试点 $3, 4$。在此总结一下，上面的这种处理区间最值的方法的核心是**数域的划分**。也就是说，我们以一个 $\log$ 的代价，将区间最值操作转化成了最值数域上的区间加减操作。

---

现在我们再考虑只有操作 $1, 3, 4, 5$ 的情况。 我们在线段树的每个节点上维护两个值：区间当前最大值 $mx$ 和历史最大值 $mx'$，以及两个标记：区间加标记 $add$ 和历史最大加减标记 $add'$。这里唯一不容易理解的就是“历史最大加减标记”，它表示的是**从上一次下传 $add$ 到现在，区间加标记达到的最大值**。

考虑历史最值标记的合并。当节点 $x$ 下传标记到它的儿子 $ch$ 时，儿子历史最大值应与当前值取 $\max$，写成公式就是下面的样子：

$$add'_{ch} \leftarrow \max(add'_{ch}, add_{ch} + add'_x)$$

$$mx'_{ch} \leftarrow \max(mx'_{ch}, mx_{ch} + add'_x)$$

这样做的正确性是显然的，区间和可以用常规方法维护，复杂度为 $O(m \log n)$。于是我们又解决了测试点 $5, 6$。

---

现在考虑将这两部分合并起来，在这里我们应用之前的思想，把一个区间的元素划分成最大值和非最大值两部分。那么区间最值操作可以转化为对最大值的加减操作，而题目中的区间加减则同时作用于这两类值。于是我们以一个 $\log$ 的代价把问题就变成了两个数域上的区间加减操作和历史最值询问。具体地说，我们需要维护四种标记：

- 最大值加减标记
- 最大值历史最大的加减标记
- 非最大值加减标记
- 非最大值历史最大的加减标记

前两个标记是只修改最大值的，所以下传时要判断当前节点是否包含区间最大值。总复杂度为 $O(m \log^2 n)$，需要注意一下常数。

如果有地方不太理解的话，可以看下面的代码：

```
#include <cstdio>
#include <cctype>
#include <algorithm>
using namespace std;
typedef long long ll;
const int MAXN=550000;
const int INF=2E9+233;
int read()
{
    int x=0, w=0; char ch=0;
    while (!isdigit(ch)) w|=ch=='-', ch=getchar();
    while (isdigit(ch)) x=(x<<3)+(x<<1)+(ch^48), ch=getchar();
    return w?-x:x;
}
struct SegmentTree
```

```
{
    struct Node
    {
        int l, r;
        int mx, mx_, se, cnt; ll sum;
        int add1, add1_, add2, add2_;
    } tr[4*MAXN];
    #define lc (o<<1)
    #define rc (o<<1|1)
    void pushup(int o)
    {
        tr[o].sum=tr[lc].sum+tr[rc].sum;
        tr[o].mx_=max(tr[lc].mx_, tr[rc].mx_);
        if (tr[lc].mx==tr[rc].mx)
        {
            tr[o].mx=tr[lc].mx;
            tr[o].se=max(tr[lc].se, tr[rc].se);
            tr[o].cnt=tr[lc].cnt+tr[rc].cnt;
        }
        else if (tr[lc].mx>tr[rc].mx)
        {
            tr[o].mx=tr[lc].mx;
            tr[o].se=max(tr[lc].se, tr[rc].mx);
            tr[o].cnt=tr[lc].cnt;
        }
        else
        {
            tr[o].mx=tr[rc].mx;
            tr[o].se=max(tr[lc].mx, tr[rc].se);
            tr[o].cnt=tr[rc].cnt;
        }
    }
    void update(int o, int k1, int k1_, int k2, int k2_)
    {
        tr[o].sum+=1ll*k1*tr[o].cnt+1ll*k2*(tr[o].r-tr[o].l+1-tr[o].cnt);
        tr[o].mx_=max(tr[o].mx_, tr[o].mx+k1_);
        tr[o].add1_=max(tr[o].add1_, tr[o].add1+k1_);
        tr[o].mx+=k1, tr[o].add1+=k1;
        tr[o].add2_=max(tr[o].add2_, tr[o].add2+k2_);
        if (tr[o].se!=-INF) tr[o].se+=k2;
        tr[o].add2+=k2;
    }
    void pushdown(int o)
    {
        int tmp=max(tr[lc].mx, tr[rc].mx);
        if (tr[lc].mx==tmp)
            update(lc, tr[o].add1, tr[o].add1_, tr[o].add2, tr[o].add2_);
        else update(lc, tr[o].add2, tr[o].add2_, tr[o].add2, tr[o].add2_);
        if (tr[rc].mx==tmp)
            update(rc, tr[o].add1, tr[o].add1_, tr[o].add2, tr[o].add2_);
        else update(rc, tr[o].add2, tr[o].add2_, tr[o].add2, tr[o].add2_);
        tr[o].add1=tr[o].add1_=tr[o].add2=tr[o].add2_=0;
    }
    void build(int o, int l, int r, int* a)
    {
        tr[o].l=l, tr[o].r=r;
```

```
        tr[o].add1=tr[o].add1_=tr[o].add2=tr[o].add2_=0;
        if (l==r)
        {
            tr[o].sum=tr[o].mx_=tr[o].mx=a[l];
            tr[o].se=-INF, tr[o].cnt=1;
            return;
        }
        int mid=l+r>>1;
        build(lc, l, mid, a);
        build(rc, mid+1, r, a);
        pushup(o);
    }
    void modify1(int o, int l, int r, int k)
    {
        if (tr[o].l>r||tr[o].r<l) return;
        if (l<=tr[o].l&&tr[o].r<=r)
            { update(o, k, k, k, k); return; }
        pushdown(o);
        modify1(lc, l, r, k), modify1(rc, l, r, k);
        pushup(o);
    }
    void modify2(int o, int l, int r, int k)
    {
        if (tr[o].l>r||tr[o].r<l||k>=tr[o].mx) return;
        if (l<=tr[o].l&&tr[o].r<=r&&k>tr[o].se)
            { update(o, k-tr[o].mx, k-tr[o].mx, 0, 0); return; }
        pushdown(o);
        modify2(lc, l, r, k), modify2(rc, l, r, k);
        pushup(o);
    }
    ll query3(int o, int l, int r)
    {
        if (tr[o].l>r||tr[o].r<l) return 0;
        if (l<=tr[o].l&&tr[o].r<=r) return tr[o].sum;
        pushdown(o);
        return query3(lc, l, r)+query3(rc, l, r);
    }
    int query4(int o, int l, int r)
    {
        if (tr[o].l>r||tr[o].r<l) return -INF;
        if (l<=tr[o].l&&tr[o].r<=r) return tr[o].mx;
        pushdown(o);
        return max(query4(lc, l, r), query4(rc, l, r));
    }
    int query5(int o, int l, int r)
    {
        if (tr[o].l>r||tr[o].r<l) return -INF;
        if (l<=tr[o].l&&tr[o].r<=r) return tr[o].mx_;
        pushdown(o);
        return max(query5(lc, l, r), query5(rc, l, r));
    }
    #undef lc
    #undef rc
} sgt;
int a[MAXN];
int main()
```

```
{
//  freopen("segbeats.in", "r", stdin);
//  freopen("segbeats.out", "w", stdout);
    int n=read(), m=read();
    for (int i=1; i<=n; i++) a[i]=read();
    sgt.build(1, 1, n, a);
    while (m--)
    {
        int op=read(), l, r, k;
        switch (op)
        {
            case 1:
                l=read(), r=read(), k=read();
                sgt.modify1(1, l, r, k);
                break;
            case 2:
                l=read(), r=read(), k=read();
                sgt.modify2(1, l, r, k);
                break;
            case 3:
                l=read(), r=read();
                printf("%lld\n", sgt.query3(1, l, r));
                break;
            case 4:
                l=read(), r=read();
                printf("%d\n", sgt.query4(1, l, r));
                break;
            case 5:
                l=read(), r=read();
                printf("%d\n", sgt.query5(1, l, r));
                break;
        }
    }
    return 0;
}
```

# 多项式

## NTT

```
int n , m;

int Pow( int x , int a ) {
    int ret = 1;
    while( a ) {
        if( a & 1 ) ret = ret * 1ll * x % P;
        x = x * 1ll * x % P , a >>= 1;
    }
    return ret;
}

int Wn[2][MAXN];
void getwn( int len ) {
    for( int mid = 1 ; mid < len ; mid <<= 1 ) {
```

```
        int w0 = Pow( 3 , ( P - 1 ) / ( mid << 1 ) ) , w1 = Pow( 3 , P - 1 - ( P
- 1 ) / ( mid << 1 ) );
        Wn[0][mid] = Wn[1][mid] = 1;
        for( int i = 1 ; i < mid ; ++ i )
            Wn[0][mid + i] = Wn[0][mid + i - 1] * 1ll * w0 % P,
            Wn[1][mid + i] = Wn[1][mid + i - 1] * 1ll * w1 % P;
    }
}
int rev[MAXN];
void getr( int len ) {
    int l = __builtin_ctz( len ) - 1;
    for( int i = 1 ; i < len ; ++ i ) rev[i] = ( rev[i >> 1] >> 1 ) | ( ( i & 1 )
<< l );
}
void NTT( int A[] , int len , int typ ) {
    for( int i = 0 ; i < len ; ++ i ) if( i < rev[i] ) swap( A[i] , A[rev[i]] );
    for( int mid = 1 ; mid < len ; mid <<= 1 )
        for( int i = 0 ; i < len ; i += ( mid << 1 ) )
            for( int j = i ; j < i + mid ; ++ j ) {
                int t0 = A[j] , t1 = A[j + mid] * 1ll * Wn[typ][j - i + mid] % P;
                A[j] = ( t0 + t1 > P ? t0 + t1 - P : t0 + t1 ) , A[j + mid] = (
t0 < t1 ? t0 - t1 + P : t0 - t1 );
            }
    if( typ ) for( int i = 0 , iv = Pow( len , P - 2 ) ; i < len ; ++ i ) A[i] =
A[i] * 1ll * iv % P;
}

int A[MAXN] , B[MAXN];
void solve() {
    cin >> n >> m;
    rep( i , 0 , n ) scanf("%d",A + i);
    rep( i , 0 , m ) scanf("%d",B + i);
    int len = 1;
    while( len <= n + m ) len <<= 1;
    getwn( len ) , getr( len );
    NTT( A , len , 0 ) , NTT( B , len , 0 );
    rep( i , 0 , len - 1 ) A[i] = A[i] * 1ll * B[i] % P;
    NTT( A , len , 1 );
    rep( i , 0 , n + m ) printf("%d ",A[i]);
}
```

## FFT

```
struct comp {
    double a , b;
    comp( double a = 0 , double b = 0 ) : a(a) , b(b) {}
}A[MAXN] , B[MAXN] , ans[MAXN];
comp operator + ( comp a , comp b ) { return comp(a.a+b.a , a.b+b.b); }
comp operator - ( comp a , comp b ) { return comp(a.a-b.a , a.b-b.b); }
comp operator * ( comp a , comp b ) { return comp(a.a*b.a-a.b*b.b ,
a.a*b.b+a.b*b.a); }

void FFT( int len , comp* A , int type ) {
    if( len == 1 ) return;
    comp a[2][len>>1];
```

```
    for( int i = 0 ; i < len ; ++ i )
        a[i&1][i>>1] = A[i];
    FFT( len>>1 , a[0] , type ) , FFT( len>>1 , a[1] , type );
    comp W( cos(2.0*pi/len) , type*sin(2.0*pi/len) ) , w(1,0);
    for( int i = 0 ; i < (len>>1) ; ++ i , w=w*W )
        A[i] = a[0][i] + w*a[1][i] ,
        A[i+(len>>1)] = a[0][i] - w*a[1][i];
}
int R[MAXN];
void FFFT( int len , comp* A , int type ) {
    for( int i = 0 ; i < len ; ++ i ) if( i < R[i] ) swap( A[i] , A[R[i]] );
    for( int l = 2 ; l <= len ; l<<=1 ) {
        comp W( cos(2*pi/l) , sin(2*pi/l)*type );
        for( int st = 0 ; st < len ; st += (l) ) {
            comp w(1,0) , t1 , t2;
            for( int j = 0 ; j < (l>>1) ; ++ j , w=w*W ) {
                t1 = A[st+j] , t2 = w*A[st+(l>>1)+j];
                A[st+j] = t1 + t2;
                A[st+j+(l>>1)] = t1 - t2;
            }
        }
    }
}

int main() {
    cin >> n >> m;
    int t , l = 0;
    for( int i = 0 ; i <= n ; ++ i )
        scanf("%d",&t) , A[i].a = 1.0*t;
    for( int i = 0 ; i <= m ; ++ i )
        scanf("%d",&t) , B[i].a = 1.0*t;
    int p = 1; while( p <= n + m ) p <<= 1 , ++ l;
    for( int i = 0 ; i < p ; ++ i ) R[i] = (R[i>>1] >> 1) | ((i&1) << l - 1);
    FFFT( p , A , 1 ) , FFFT( p , B , 1 );
    for( int i = 0 ; i <= (p) ; ++ i )
        ans[i] = A[i] * B[i];
    FFFT( p , ans , -1 );
    for( int i = 0 ; i <= n + m ; ++ i )
        printf("%d ",(int)(ans[i].a/(1.0*p)+0.5));
}
```

## MTT

```
namespace bf {
const int P[] = { 998244353 , 1004535809 , 469762049 };
const ll M1 = P[0] * 1ll * P[1];
const ll M[3] = { P[1] * 1ll * P[2] , P[0] * 1ll * P[2] , P[0] * 1ll * P[1] };

int Pow( int x , int a , int p ) {
    int ret = 1;
    while( a ) {
        if( a & 1 ) ret = ret * 1ll * x % p;
        x = x * 1ll * x % p , a >>= 1;
    }
    return ret;
```

```
}
const int iv1 = Pow( P[0] , P[1] - 2 , P[1] ) , iv2 = Pow( M1 % P[2] , P[2] - 2 ,
P[2] );

struct mint {
    int a , b , c;
    mint( ) : a(0) , b(0) , c(0) {}
    mint( int a , int b , int c ) : a(a) , b(b) , c(c) {}
    mint( int a ) : a(a) , b(a) , c(a) {}
    mint operator + ( mint x ) {
        return mint( ( a + x.a ) % P[0] , ( b + x.b ) % P[1] , ( c + x.c ) % P[2]
);
    }
    mint operator - ( mint x ) {
        return mint( ( a + P[0] - x.a ) % P[0] , ( b + P[1] - x.b ) % P[1] , ( c
+ P[2] - x.c ) % P[2] );
    }
    mint operator - ( int x ) {
        return mint( ( a + P[0] - x ) % P[0] , ( b + P[1] - x ) % P[1] , ( c +
P[2] - x ) % P[2] );
    }
    mint operator * ( mint x ) {
        return mint( a * 1ll * x.a % P[0] , b * 1ll * x.b % P[1] , c * 1ll * x.c
% P[2] );
    }
    mint operator / ( int x ) {
        return mint( a / x , b / x , c / x );
    }
    int tp( ) {
        ll x = ( b - a + P[1] ) % P[1] * 1ll * iv1 % P[1] * P[0] + a;
        return ( ( c - x % P[2] + P[2] ) % P[2] * iv2 % P[2] * ( M1 % p ) % p + x
) % p;
    }
} pp( P[0] , P[1] , P[2] );

mint Pow( mint x , mint a ) {
    return mint( Pow( x.a , a.a , P[0] ) , Pow( x.b , a.b , P[1] ) , Pow( x.c ,
a.c , P[2] ) );
}

mint Wn[2][1200000];
void getwn( int len ) {
    for( int mid = 1 ; mid < len ; mid <<= 1 ) {
        mint w0 = Pow( mint( 3 ) , ( pp - mint( 1 ) ) / ( mid << 1 ) ) , w1 =
Pow( mint( 3 ) , ( pp - 1 ) - ( pp - mint( 1 ) ) / ( mid << 1 ) );
        Wn[0][mid] = Wn[1][mid] = mint( 1 );
        for( int i = 1 ; i < mid ; ++ i )
            Wn[0][mid + i] = Wn[0][mid + i - 1] * w0,
            Wn[1][mid + i] = Wn[1][mid + i - 1] * w1;
    }
}
int rev[MAXN];
void getr( int len ) {
    int l = __builtin_ctz( len ) - 1;
    for( int i = 1 ; i < len ; ++ i ) rev[i] = ( rev[i >> 1] >> 1 ) | ( ( i & 1 )
<< l );
```

```
}
void NTT( mint A[] , int len , int typ ) {
    for( int i = 0 ; i < len ; ++ i ) if( i < rev[i] ) swap( A[i] , A[rev[i]] );
    for( int mid = 1 ; mid < len ; mid <<= 1 )
        for( int i = 0 ; i < len ; i += ( mid << 1 ) )
            for( int j = i ; j < i + mid ; ++ j ) {
                mint t0 = A[j] , t1 = A[j + mid] * Wn[typ][j - i + mid];
                A[j] = t0 + t1 , A[j + mid] = t0 - t1;
            }
    if( typ ) {
        mint iv = Pow( mint(len) , mint( pp - 2 ) );
        for( int i = 0 ; i < len ; ++ i ) A[i] = A[i] * iv;
    }
}
mint A[MAXN] , B[MAXN];
void mul( int a[] , int b[] , int c[] , int n , int m ) {
    int len = 1;
    while( len <= n + m + 1 ) len <<= 1;
    getr( len );
    rep( i , 0 , len - 1 ) A[i] = mint( a[i] ) , B[i] = mint( b[i] );
//  NTT( A , len , 0 ) , NTT( A , len , 1 );
//  rep( i , 0 , len - 1 ) {
//      printf("%d\n",A[i].tp());
//  }
//  return;
    NTT( A , len , 0 ) , NTT( B , len , 0 );
    rep( i , 0 , len - 1 ) A[i] = A[i] * B[i];
    NTT( A , len , 1 );
    rep( i , 0 , len - 1 )
        c[i] = A[i].tp();
}
}

int A[MAXN] , B[MAXN] , C[MAXN];

void solve() {
    bf::getwn( 1 << 19 );
    cin >> n >> m >> p;
    rep( i , 0 , n ) scanf("%d",A + i) , A[i] %= p;
    rep( i , 0 , m ) scanf("%d",B + i) , B[i] %= p;
    bf::mul( A , B , C , n , m );
    rep( i , 0 , n + m )
        printf("%d ",C[i]);
}
```

## 分治 NTT

```
#include "iostream"
#include "algorithm"
#include "cstring"
#include "cstdio"
#include "cmath"
#include "vector"
#include "map"
#include "set"
```

```cpp
#include "queue"
using namespace std;
#define MAXN 400006
//#define int long long
#define rep(i, a, b) for (int i = (a), i##end = (b); i <= i##end; ++i)
#define per(i, a, b) for (int i = (a), i##end = (b); i >= i##end; --i)
#define pii pair<int,int>
#define fi first
#define se second
#define mp make_pair
#define pb push_back
#define eb emplace_back
#define vi vector<int>
#define all(x) (x).begin() , (x).end()
#define mem( a ) memset( a , 0 , sizeof a )
typedef long long ll;
const int P = 998244353;
int n , m;

int Pow( int x , int a ) {
    int ret = 1;
    while( a ) {
        if( a & 1 ) ret = ret * 1ll * x % P;
        x = x * 1ll * x % P , a >>= 1;
    }
    return ret;
}

int Wn[2][540000];
void getwn( int len ) {
    for( int mid = 1 ; mid < len ; mid <<= 1 ) {
        int w0 = Pow( 3 , ( P - 1 ) / ( mid << 1 ) ) , w1 = Pow( 3 , P - 1 - ( P
- 1 ) / ( mid << 1 ) );
        Wn[0][mid] = Wn[1][mid] = 1;
        for( int i = 1 ; i < mid ; ++ i )
            Wn[0][mid + i] = Wn[0][mid + i - 1] * 1ll * w0 % P,
                    Wn[1][mid + i] = Wn[1][mid + i - 1] * 1ll * w1 % P;
    }
}
int rev[MAXN];
void getr( int len ) {
    int l = __builtin_ctz( len ) - 1;
    for( int i = 1 ; i < len ; ++ i ) rev[i] = ( rev[i >> 1] >> 1 ) | ( ( i & 1 )
<< l );
}
void NTT( int A[] , int len , int typ ) {
    for( int i = 0 ; i < len ; ++ i ) if( i < rev[i] ) swap( A[i] , A[rev[i]] );
    for( int mid = 1 ; mid < len ; mid <<= 1 )
        for( int i = 0 ; i < len ; i += ( mid << 1 ) )
            for( int j = i ; j < i + mid ; ++ j ) {
                int t0 = A[j] , t1 = A[j + mid] * 1ll * Wn[typ][j - i + mid] % P;
                A[j] = ( t0 + t1 > P ? t0 + t1 - P : t0 + t1 ) , A[j + mid] = (
t0 < t1 ? t0 - t1 + P : t0 - t1 );
            }
    if( typ ) for( int i = 0 , iv = Pow( len , P - 2 ) ; i < len ; ++ i ) A[i] =
A[i] * 1ll * iv % P;
```

```
}

int f[MAXN] , g[MAXN];

int tA[MAXN] , tB[MAXN];
void cdq( int A[] , int B[] , int l , int r ) {
    if( l == r ) return;
    int m = l + r >> 1;
    cdq( A , B , l , m );
    rep( j , 0 , m - l ) tA[j] = A[j + l];
    rep( j , 1 , r - l ) tB[j] = B[j];
    int len = 1;
    while( len <= ( r - l ) * 2 ) len <<= 1;
    rep( j , m - l + 1 , len - 1 ) tA[j] = 0;
    rep( j , r - l + 1 , len - 1 ) tB[j] = 0;
    getr( len );
    NTT( tA , len , 0 ) , NTT( tB , len , 0 );
    rep( i , 0 , len - 1 ) tA[i] = tA[i] * 1ll * tB[i] % P;
    NTT( tA , len , 1 );
    rep( i , m - l + 1 , r - l ) A[i + l] = ( A[i + l] + tA[i] ) % P;
    cdq( A , B , m + 1 , r );
}

void solve() {
    getwn( 1 << 19 );
    cin >> n;
    rep( i , 1 , n - 1 ) scanf("%d",g + i);
    f[0] = 1;
    cdq( f , g , 0 , n - 1 );
    rep( i , 0 , n - 1 ) printf("%d ",f[i]);
}

signed main() {
//    int T;cin >> T;while( T-- ) solve();
    solve();
}
```

## 求逆

```
int tmpa[MAXN];
void Inv( int A[] , int B[] , int n ) { // B = inv A
    if( n == 1 ) return void( B[0] = Pow( A[0] , P - 2 ) );
    Inv( A , B , n + 1 >> 1 );
    int len = 1;
    while( len <= n + n ) len <<= 1;
    getr( len );
    for( int i = 0 ; i < n ; ++ i ) tmpa[i] = A[i];
    for( int i = n ; i < len ; ++ i ) tmpa[i] = 0;
    NTT( tmpa , len , 0 ) , NTT( B , len , 0 );
    rep( i , 0 , len - 1 ) B[i] = B[i] * ( 2 - tmpa[i] * 1ll * B[i] % P + P ) %
P;
    NTT( B , len , 1 );
    rep( i , n , len - 1 ) B[i] = 0;
}
```

```
int A[MAXN] , B[MAXN];

void solve() {
    getwn( ( 1 << 18 ) );
    cin >> n;
    rep( i , 0 , n - 1 ) scanf("%d",A + i);
    Inv( A , B , n );
    rep( i , 0 , n - 1 ) printf("%d ",B[i]);
}
```

## ln

$$-\ln(1-x) = x + \frac{x^2}{2} + \frac{x^3}{3} + \cdots$$

```
int tmpa[MAXN];
void Inv( int A[] , int B[] , int n ) { // B = inv A
    if( n == 1 ) return void( B[0] = Pow( A[0] , P - 2 ) );
    Inv( A , B , n + 1 >> 1 );
    int len = 1;
    while( len <= n + n ) len <<= 1;
    getr( len );
    for( int i = 0 ; i < n ; ++ i ) tmpa[i] = A[i];
    for( int i = n ; i < len ; ++ i ) tmpa[i] = 0;
    NTT( tmpa , len , 0 ) , NTT( B , len , 0 );
    rep( i , 0 , len - 1 ) B[i] = B[i] * ( 2 - tmpa[i] * 1ll * B[i] % P + P ) %
P;
    NTT( B , len , 1 );
    rep( i , n , len - 1 ) B[i] = 0;
}

int tmp[MAXN] , iv[MAXN];
void Ln( int A[] , int B[] , int n ) {
    rep( i , 0 , n - 2 ) tmp[i] = A[i + 1] * 1ll * ( i + 1 ) % P;
    Inv( A , iv , n );
    int len = 1;
    while( len <= n + n ) len <<= 1;
    NTT( tmp , len , 0 ) , NTT( iv , len , 0 );
    rep( i , 0 , len - 1 ) B[i] = tmp[i] * 1ll * iv[i] % P;
    NTT( B , len , 1 );
    per( i , len - 1 , 1 ) B[i] = B[i - 1] * 1ll * Pow( i , P - 2 ) % P;
    B[0] = 0;
}

int A[MAXN] , B[MAXN];

void solve() {
    getwn( ( 1 << 18 ) );
    cin >> n;
    rep( i , 0 , n - 1 ) scanf("%d",A + i);
    Ln( A , B , n );
    rep( i , 0 , n - 1 ) printf("%d ",B[i]);
```

```
}
```

## exp

可以递推，类似于：

### D AUOJ117 随机变量

不难发现，设 $A(z)=\sum_{i=0}^{k}p_kz^k$，然后只要求出 $A^n(z)\pmod{z^x}$，就可以算出答案（分别算最后得值小于和大于等于 $x$ 得情况）。

这个形式的式子是可以类似暴力多项式 exp 来做的，设 $G(z)=A^n(z)$，那么我们对两边分别求导，得到

$$\begin{aligned}G'(z)&=nA'(z)A^{n-1}(z)\\G'(z)A(z)&=nA'(z)A^n(z)\\G'(z)A(z)&=nA'(z)G(z)\\\forall t\le x,\sum_{i+1+j=t}z^tg_iia_j&=n\sum_{i+1+j=t}a_iig_j\\\sum_{i=0}^{t}g_iia_{t-i}&=n\sum_{i=0}^{t}g_ia_{t-i}(t-i)\\\sum_{i=0}^{t}g_i\left[ia_{t-i}-na_{t-i}(t-i)\right]&=0\end{aligned}$$

最后得到的这个式子就可以 $O(k)$ 递推了，因为 $t-i$ 的范围是 $\le k$，然后复杂度就是 $O(xk)$ 大概 $5\times10^7$。

正常做法：

**问题**

**已知 $G[F(x)]\equiv0\pmod{x^n}$，给出 $G(x)$，求 $F(x)$**

- **记 $F_t(x)$ 表示经过 $t$ 次迭代之后的 $F(x)$，满足**

$$G[F_t(x)]\equiv0\pmod{x^{2^t}}$$

- **考虑泰勒展开公式，对于函数 $H(x)$，选择展开点 $x_0$，那么有**

$$H(x)=\sum_{i\ge0}\frac{H^{(i)}(x_0)}{i!}(x-x_0)^i$$

- 考虑将 $G[F_{t+1}(x)]$ 从 $F_t(x)$ 处展开

$$
\begin{aligned}
G[F_{t+1}(x)] &= \sum_{i\geq 0} \frac{G^{(i)}[F_t(x)]}{i!}[F_{t+1}(x)-F_t(x)]^i \\
&= G[F_t(x)] + G'[F_t(x)] \times [F_{t+1}(x)-F_t(x)] + \sum_{i\geq 2} \cdots \\
0 &\equiv G[F_t(x)] + G'[F_t(x)] \times [F_{t+1}(x)-F_t(x)] \pmod{x^{2^{t+1}}} \\
F_{t+1}(x) &\equiv F_t(x) - \frac{G[F_t(x)]}{G'[F_t(x)]} \pmod{x^{2^{t+1}}}
\end{aligned}
$$

- 注意这里的 $G'[F_t(x)]$ 是先求出 $G'(x)$，再代入 $F_t(x)$，并非对 $G[F_t(x)]$ 求导

```
int tmpa[MAXN];
void Inv( const int A[] , int B[] , int n ) { // B = inv A
    if( n == 1 ) return void( B[0] = Pow( A[0] , P - 2 ) );
    Inv( A , B , n + 1 >> 1 );
    int len = 1;
    while( len <= n + n ) len <<= 1;
    getr( len );
    for( int i = 0 ; i < n ; ++ i ) tmpa[i] = A[i];
    for( int i = n ; i < len ; ++ i ) tmpa[i] = 0;
    NTT( tmpa , len , 0 ) , NTT( B , len , 0 );
    rep( i , 0 , len - 1 ) B[i] = B[i] * ( 2 - tmpa[i] * 1ll * B[i] % P + P ) %
P;
    NTT( B , len , 1 );
    rep( i , n , len - 1 ) B[i] = 0;
}

int tmp[MAXN] , iv[MAXN];
void Ln( const int A[] , int B[] , int n ) {
    rep( i , 0 , n - 1 ) tmp[i] = A[i + 1] * 1ll * ( i + 1 ) % P;
    Inv( A , iv , n );
//  printf("After Inv: "); rep( i , 0 , n - 1 ) printf("%d ",iv[i]); puts("");
    int len = 1;
    while( len <= n + n ) len <<= 1;
    rep( i , n , len - 1 ) tmp[i] = 0;
    NTT( tmp , len , 0 ) , NTT( iv , len , 0 );
    rep( i , 0 , len - 1 ) B[i] = tmp[i] * 1ll * iv[i] % P;
    NTT( B , len , 1 );
    per( i , n - 1 , 1 ) B[i] = B[i - 1] * 1ll * Pow( i , P - 2 ) % P;
    B[0] = 0;
    rep( i , n , len - 1 ) B[i] = 0;
    rep( i , 0 , len - 1 ) iv[i] = 0;
}

int tmpp[MAXN];
void Exp( const int A[] , int B[] , int n ) {
    if( n == 1 ) return void( B[0] = 1 );
    Exp( A , B , n + 1 >> 1 );
```

```
    int len = 1;
    while( len <= n + n ) len <<= 1;
    getr( len );
    rep( i , 0 , len - 1 ) tmpp[i] = 0;
    Ln( B , tmpp , n );
//  printf("After Ln: "); rep( i , 0 , n - 1 ) printf("%d ",tmpp[i]); puts("");
    rep( i , 0 , n - 1 ) tmpp[i] = ( A[i] - tmpp[i] < 0 ? A[i] - tmpp[i] + P :
A[i] - tmpp[i] );
    ++ tmpp[0];
    NTT( B , len , 0 ) , NTT( tmpp , len , 0 );
    for( int i = 0 ; i < len ; ++ i ) B[i] = B[i] * 1ll * tmpp[i] % P;
    NTT( B , len , 1 );
    for( int i = n ; i < len ; ++ i ) B[i] = 0;
//  rep( i , 0 , n ) printf("%d ",B[i]); puts("");
}

int A[MAXN] , B[MAXN];

void solve() {
    getwn( ( 1 << 18 ) );
    cin >> n;
    rep( i , 0 , n - 1 ) scanf("%d",A + i);
    Exp( A , B , n );
    rep( i , 0 , n - 1 ) printf("%d ",B[i]);
}
```

## vector 乘 + 多点求值

```
int Pow( int x , int a ) {
    int ret = 1;
    while( a ) {
        if( a & 1 ) ret = ret * 1ll * x % P;
        x = x * 1ll * x % P , a >>= 1;
    }
    return ret;
}

int Wn[2][200006];
void getwn( int len ) {
    for( int mid = 1 ; mid < len ; mid <<= 1 ) {
        int w0 = Pow( 3 , ( P - 1 ) / ( mid << 1 ) ) , w1 = Pow( 3 , P - 1 - ( P
- 1 ) / ( mid << 1 ) );
        Wn[0][mid] = Wn[1][mid] = 1;
        for( int i = 1 ; i < mid ; ++ i )
            Wn[0][mid + i] = Wn[0][mid + i - 1] * 1ll * w0 % P,
                    Wn[1][mid + i] = Wn[1][mid + i - 1] * 1ll * w1 % P;
    }
}
int rev[MAXN];
void getr( int len ) {
    int l = __builtin_ctz( len ) - 1;
    for( int i = 1 ; i < len ; ++ i ) rev[i] = ( rev[i >> 1] >> 1 ) | ( ( i & 1 )
<< l );
}
void NTT( vector<int>& A , int len , int typ ) { // A.size() == len
```

```
    for( int i = 0 ; i < len ; ++ i ) if( i < rev[i] ) swap( A[i] , A[rev[i]] );
    for( int mid = 1 ; mid < len ; mid <<= 1 )
        for( int i = 0 ; i < len ; i += ( mid << 1 ) )
            for( int j = i ; j < i + mid ; ++ j ) {
                int t0 = A[j] , t1 = A[j + mid] * 1ll * Wn[typ][j - i + mid] % P;
                A[j] = ( t0 + t1 > P ? t0 + t1 - P : t0 + t1 ) , A[j + mid] = (
t0 < t1 ? t0 - t1 + P : t0 - t1 );
            }
    if( typ ) for( int i = 0 , iv = Pow( len , P - 2 ) ; i < len ; ++ i ) A[i] =
A[i] * 1ll * iv % P;
}

void mul( vi& ret , vi& a , vi& b , int flg = 0 ) {
    int len = 1 , as = a.size() , bs = b.size();
    while( len <= as + bs ) len <<= 1;
    getr( len );
    a.resize( len ) , b.resize( len ) , ret.resize( len );
    NTT( a , len , 0 ) , NTT( b , len , 0 );
    rep( i , 0 , len - 1 ) ret[i] = a[i] * 1ll * b[i] % P;
    NTT( ret , len , 1 );
    ret.resize( as + bs - 1 );
    if( flg ) {
        NTT( a , len , 1 ) , NTT( b , len , 1 );
        a.resize( as ) , b.resize( bs );
    }
}
void mult( vi& ret , vi& a , vi& b , int flg = 0 ) {
    int as = a.size() , bs = b.size();
//  cout << as << ' ' << bs << endl;
    reverse( all( b ) );
    mul( ret , a , b , flg );
    rep( i , 0 , as - 1 ) ret[i] = ret[i + bs - 1];
    ret.resize( as );
    if( flg ) {
        reverse( all( b ) );
    }
}

vi tmpa;
void Inv( const vi& A , vi& B , int n ) {
    if( n == 1 ) return void( B[0] = Pow( A[0] , P - 2 ) );
    Inv( A , B , n + 1 >> 1 );
    int len = 1;
    while( len <= n + n ) len <<= 1;
    getr( len );
    tmpa.resize( len ) , B.resize( len );
    rep( i , 0 , n - 1 ) tmpa[i] = A[i];
    rep( i , n , len - 1 ) tmpa[i] = 0;
    NTT( tmpa , len , 0 ) , NTT( B , len , 0 );
    rep( i , 0 , len - 1 ) B[i] = B[i] * ( 2 - tmpa[i] * 1ll * B[i] % P + P ) %
P;
    NTT( B , len , 1 );
    B.resize( n );
}

vector<int> M[MAXN << 2];
```

```
void init( int rt , int l , int r ) {
    if( l == r ) {
        M[rt].pb( 1 ) , M[rt].pb( P - A[l] );
        return;
    }
    int m = l + r >> 1;
    init( rt << 1 , l , m ) , init( rt << 1 | 1 , m + 1 , r );
    mul( M[rt] , M[rt << 1] , M[rt << 1 | 1] , 1 );
}

int ans[MAXN];

void solve( vi& F , int rt , int l , int r ) {
    if( l == r ) {
        ans[l] = F[0];
        return;
    }
    int m = l + r >> 1;
    F.resize( r - l + 1 );
    vi tmp;
    mult( tmp , F , M[rt << 1 | 1] , 1 );
    solve( tmp , rt << 1 , l , m );
    mult( tmp , F , M[rt << 1] , 1 );
    solve( tmp , rt << 1 | 1 , m + 1 , r );
}

void solve() {
    getwn( 1 << 17 );
    cin >> n >> m;
    vi f(n + 1) , tmp( 1 ) , t;
    rep( i , 0 , n ) scanf("%d",&f[i]);
    rep( i , 1 , m ) scanf("%d",A + i);
    init( 1 , 1 , m );
    int l = M[1].size();
    M[1].resize( ( 1 << 17 ) );
    Inv( M[1] , tmp , n + 1 );
    M[1].resize( l );
    mult( t , f , tmp , 1 );
    solve( t , 1 , 1 , m );
    rep( i , 1 , m ) printf("%d\n",ans[i]);
}
```

# 除法

具体来说，设多项式 $A$ 为 $n$ 次多项式，考虑一种操作 $R$，使得

$$A_R(x) = x^n A(\frac{1}{x})$$

稍微想象一下，可以发现 $A_R[i] = A[n-i]$（$[i]$ 表示多项式的第 $i$ 次系数）。

这个操作可以 $O(n)$ 完成。

然后开始化式子。

$$F(x) = Q(x) * G(x) + R(x)$$

$$F(\frac{1}{x}) = Q(\frac{1}{x}) * G(\frac{1}{x}) + R(\frac{1}{x})$$

$$x^n F(\frac{1}{x}) = x^{n-m} Q(\frac{1}{x}) * x^m G(\frac{1}{x}) + x^{n-m+1} * x^{m-1} R(\frac{1}{x})$$

$$F_R(x) = Q_R(x) * G_R(x) + x^{n-m+1} * R_R(x)$$

$$F_R(x) \equiv Q_R(x) * G_R(x) + x^{n-m+1} * R_R(x) \pmod{x^{n-m+1}}$$

$$F_R(x) \equiv Q_R(x) * G_R(x) \pmod{x^{n-m+1}}$$

$$Q_R(x) \equiv F_R(x) * G_R^{-1}(x) \pmod{x^{n-m+1}}$$

求一遍 $G_R$ 的逆，然后就可以利用多项式乘法求出 $Q$。然后

$$R(x) = F(x) - G(x) * Q(x)$$

直接计算即可。时间复杂度$O(n \log n)$。

```
int tmpa[MAXN];
void Inv( int A[] , int B[] , int n ) { // B = inv A
    if( n == 1 ) return void( B[0] = Pow( A[0] , P - 2 ) );
    Inv( A , B , n + 1 >> 1 );
    int len = 1;
    while( len <= n + n ) len <<= 1;
    getr( len );
    for( int i = 0 ; i < n ; ++ i ) tmpa[i] = A[i];
    for( int i = n ; i < len ; ++ i ) tmpa[i] = 0;
    NTT( tmpa , len , 0 ) , NTT( B , len , 0 );
    rep( i , 0 , len - 1 ) B[i] = B[i] * ( 2 - tmpa[i] * 1ll * B[i] % P + P ) %
P;
    NTT( B , len , 1 );
    rep( i , n , len - 1 ) B[i] = 0;
}

int A[MAXN] , B[MAXN] , ivb[MAXN] , F[MAXN] , R[MAXN];

void solve() {
    getwn( ( 1 << 18 ) );
    cin >> n >> m;
    rep( i , 0 , n ) scanf("%d",&A[n - i]);
    rep( i , 0 , m ) scanf("%d",&B[m - i]);
    Inv( B , ivb , n - m + 1 );
    int len = 1;
```

```
    while( len <= n + n ) len <<= 1;
    getr( len );
    NTT( A , len , 0 ) , NTT( ivb , len , 0 );
    rep( i , 0 , len - 1 ) F[i] = 1ll * A[i] * ivb[i] % P;
    NTT( F , len , 1 ) , NTT( A , len , 1 );
    reverse( F , F + n - m + 1 ) , reverse( A , A + n + 1 ) , reverse( B , B + m
+ 1 );
    rep( i , 0 , n - m ) printf("%d ",F[i]); puts("");
    rep( i , n - m + 1 , len ) F[i] = 0;
    NTT( F , len , 0 ) , NTT( B , len , 0 );
    rep( i , 0 , len - 1 ) B[i] = B[i] * 1ll * F[i] % P;
    NTT( B , len , 1 );
    rep( i , 0 , m - 1 ) printf("%d ",( A[i] - B[i] + P ) % P );
}
```

## 组合恒等式

$$\binom{n}{m} = \binom{n}{n-m}$$

$$\sum_{i=0}^{n} \binom{n}{i} = 2^n$$

$$\sum_{i=0}^{n} \binom{n}{i} [2 \mid i] = \sum_{i=0}^{n} \binom{n}{i} [2 \nmid i] = 2^{n-1}$$

$$\sum_{i=0}^{m} \binom{n+i}{n} = \binom{n+m+1}{m}$$

$$\binom{n}{m}\binom{m}{k} = \binom{n}{k}\binom{n-k}{m-k}$$

$$\sum_{i=0}^{k} \binom{n}{i}\binom{m}{k-i} = \binom{n+m}{k}$$

## 斯特林反演

$$[n=m]=\sum_{k=m}^{n}(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}\begin{Bmatrix}k\\m\end{Bmatrix}$$

$$[n=m]=\sum_{k=m}^{n}(-1)^{n-k}\begin{Bmatrix}n\\k\end{Bmatrix}\begin{bmatrix}k\\m\end{bmatrix}$$

$$f(n)=\sum_{k=0}^{n}\begin{Bmatrix}n\\k\end{Bmatrix}g(k)\Leftrightarrow g(n)=\sum_{k=0}^{n}(-1)^{n-k}\begin{bmatrix}n\\k\end{bmatrix}f(k)$$

$$f(n)=\sum_{k=0}^{n}\begin{bmatrix}n\\k\end{bmatrix}g(k)\Leftrightarrow g(n)=\sum_{k=0}^{n}(-1)^{n-k}\begin{Bmatrix}n\\k\end{Bmatrix}f(k)$$

$$f(n)=\sum_{k\geq n}\begin{Bmatrix}k\\n\end{Bmatrix}g(k)\Leftrightarrow g(n)=\sum_{k\geq n}(-1)^{k-n}\begin{bmatrix}k\\n\end{bmatrix}f(k)$$

## Bluestein

- 任意长度 DFT
- 在 DFT 的过程中，我们想要求出原多项式在 $\omega_n^k$ 处的点值

$$y_k=\sum_{i=0}^{n-1}a_i\omega_n^{ki}$$

- $ki$ 没法用卷积表示，可以考虑将其拆成 $\frac{k^2+i^2-(k-i)^2}{2}$

$$\begin{aligned}y_k&=\sum_{i=0}^{n-1}a_i\omega_{2n}^{k^2+i^2-(k-i)^2}\\&=\omega_{2n}^{k^2}\sum_{i=0}^{n-1}a_i\omega_{2n}^{i^2}\times\omega_{2n}^{-(k-i)^2}\end{aligned}$$

更多时候拆 $\binom{k}{2}+\binom{t}{2}-\binom{k+t}{2}$

## 单位根反演

$$[n \mid k] = \frac{1}{n}\sum_{i=0}^{n-1}(\omega_n^k)^i$$

## 复合逆

可能更好的体验

对于满足 $G(F(x)) = x$ 且 $F(x)$ 的常数项为 $0$，一次项有逆元的多项式 $F(x), G(x)$，有拉格朗日反演公式：

$$[x^n]G(x) = \frac{1}{n}[x^{n-1}](\frac{x}{F(x)})^n$$

这个可以求单项系数。我们要求所有系数。

按多项式复合的做法来就可以了。

令 $L = \sqrt{n}$。

那么有：

$$G(x) = \sum_{i=1}^{n}\left(\frac{1}{i}[x^{i-1}](\frac{x}{F(x)})^i\right)x^i$$

$$= \sum_{i=0}^{L-1}\sum_{j=1}^{L}\left(\frac{1}{iL+j}[x^{iL+j-1}](\frac{x}{F(x)})^{iL+j}\right)x^{iL+j}$$

$$= \sum_{i=0}^{L-1}\sum_{j=1}^{L}\left(\frac{1}{iL+j}[x^{iL+j-1}](\frac{x}{F(x)})^{iL}(\frac{x}{F(x)})^{j}\right)x^{iL+j}$$

其中，$(\frac{x}{F(x)})^{iL}$ 和 $(\frac{x}{F(x)})^{j}$ 我们可以直接 FFT 预处理。时间复杂度 $O(n\sqrt{n}\log n)$。

观察 $[x^{iL+j-1}](\frac{x}{F(x)})^{iL}(\frac{x}{F(x)})^{j}$，这部分相当于给出两个多项式，求它们的乘积的某一项系数。这个可以 $O(n)$ 暴力求得。常数很小。

总时间复杂度 $O(n^2 + n\sqrt{n}\log n)$，可以通过。

```
#include<iostream>
#include<algorithm>
#include<cmath>
#define lg2(x)(31-__builtin_clz((x)))
using namespace std;
typedef long long LL;
const int N=32769,BS=140,md=998244353;
int n,a[N],b[N],rev[N],lim,LG,L;
int G[15][N];
int B[BS][N],B_[BS][N];
```

```
inline void upd(int&a){a+=a>>31&md;}
inline int pow(int a,int b){
    int ret=1;
    for(;b;b>>=1,a=(LL)a*a%md)if(b&1)ret=(LL)ret*a%md;
    return ret;
}
void init(int n){
    int l=-1;
    for(lim=1;lim<n;lim<<=1)++l;LG=l+1;
    for(int i=1;i<lim;++i)
    rev[i]=(rev[i>>1]>>1)|((i&1)<<l);
}
void FFT(int*a,int f){
    for(int i=1;i<lim;++i)
    if(i<rev[i])swap(a[i],a[rev[i]]);
    for(int i=0;i<LG;++i){
        const int*g=G[i],c=1<<i;
        for(int j=0;j<lim;j+=c<<1)
        for(int k=0;k<c;++k){
            const int x=a[j+k],y=g[k]*(LL)a[j+k+c]%md;
            upd(a[j+k]+=y-md),upd(a[j+k+c]=x-y);
        }
    }
    if(!f){
        const int iv=pow(lim,md-2);
        for(int i=0;i<lim;++i)a[i]=(LL)a[i]*iv%md;
        reverse(a+1,a+lim);
    }
}
void INV(int*a,int*B,int n){
    if(n==1){
        *B=pow(*a,md-2);
        return;
    }
    INV(a,B,n+1>>1);
    init(n<<1);
    static int A[N];
    for(int i=0;i<n;++i)A[i]=a[i];
    for(int i=n;i<lim;++i)A[i]=0;
    FFT(A,1),FFT(B,1);
    for(int i=0;i<lim;++i)B[i]=B[i]*(2-B[i]*(LL)A[i]%md+md)%md;
    FFT(B,0);
    for(int i=n;i<lim;++i)B[i]=0;
}
int main(){
    ios::sync_with_stdio(0),cin.tie(0),cout.tie(0);
    cin>>n>>a[0];--n;
    for(int i=0;i<15;++i){
        int*g=G[i];
        g[0]=1;
        const int gi=pow(3,(md-1)/(1<<i+1));
        for(int j=1;j<1<<i;++j)
        g[j]=(LL)g[j-1]*gi%md;
    }
    for(int i=0;i<n;++i)cin>>a[i];
    cout.put('0');
```

```
    INV(a,b,n);
    L=sqrt(n)+1;
    init(n<<1);
    **B=1;
    FFT(*B,1);
    for(int i=0;i<lim;++i)B_[0][i]=B[0][i];
    for(int i=0;i<n;++i)B[1][i]=b[i];
    FFT(B[1],1);
    for(int i=2;i<=L;++i){
        for(int j=0;j<lim;++j)
        B[i][j]=(LL)B[i-1][j]*B[1][j]%md;
        FFT(B[i],0);
        for(int j=n;j<lim;++j)B[i][j]=0;
        FFT(B[i],1);
    }
    for(int i=0;i<lim;++i)B_[0][i]=B[0][i],B_[1][i]=B[L][i];
    for(int i=2;i<=L;++i){
        for(int j=0;j<lim;++j)
        B_[i][j]=(LL)B_[i-1][j]*B_[1][j]%md;
        FFT(B_[i],0);
        for(int j=n;j<lim;++j)B_[i][j]=0;
        FFT(B_[i],1);
    }
    for(int i=0;i<=L;++i)FFT(B[i],0),FFT(B_[i],0);
    for(int i=0;i<L;++i){
        int*g_=B_[i];
        for(int j=1;j<=L;++j){
            const int x=i*L+j-1;
            if(x>=n)return 0;
            int*g=B[j];
            int ans=0;
            for(register int t=0;t<=x;++t)
            ans=(ans+(LL)g_[t]*g[x-t])%md;
            cout<<' '<<(LL)ans*pow(x+1,md-2)%md;
        }
    }
    return 0;
}
```

## 复合函数

令$L=\sqrt{n}$。

$$\sum_{i=0}^{n}[x^i]F(x)G(x)^i=\sum_{i=0}^{L-1}\sum_{j=0}^{L-1}[x^{iL+j}]F(x)G(x)^{iL+j}$$

$$=\sum_{i=0}^{L-1}G(x)^{iL}\sum_{j=0}^{L-1}[x^{iL+j}]F(x)G(x)^j$$

预处理出所有$G(x)^{iL}$和$G(x)^j$，这部分用NTT搞，时间复杂度$O(n\sqrt{n}\log n)$。

然后，对于$\sum\limits_{j=0}^{L-1}[x^{iL+j}]F(x)G(x)^j$部分暴力计算。把计算结果再乘上$G(x)^{iL}$即可。

总时间复杂度$O(n^2+n\sqrt{n}\log n)$。

实际上，$O(n^2)$部分常数小完全不虚，NTT部分需要常数比较优秀的实现。

~~听说有人用Brent-Kung算法TLE了~~

~~我复杂度比你劣但我跑得比你快~~

```
#include<cstdio>
#include<algorithm>
#include<cmath>
#include<ctime>
const int N=65536,md=998244353;
typedef long long LL;
inline void upd(int&a){a+=a>>31&md;}
inline int pow(int a,int b){
    int ret=1;
    for(;b;b>>=1,a=(LL)a*a%md)if(b&1)ret=(LL)ret*a%md;
    return ret;
}
int n,m,A[N],B[N],L,g[17][N],lim,rev[N],M,B1[150][N],B2[250][N],R[N];
void init(int n){
    int l=-1;
    for(lim=1;lim<n;lim<<=1)++l;M=l+1;
    for(int i=1;i<lim;++i)
    rev[i]=((rev[i>>1])>>1)|((i&1)<<l);
}
void NTT(int*a,int f){
    for(int i=1;i<lim;++i)if(i<rev[i])std::swap(a[i],a[rev[i]]);
    for(int i=0;i<M;++i){
        const int*G=g[i],c=1<<i;
        for(int j=0;j<lim;j+=c<<1)
        for(int k=0;k<c;++k){
            const int x=a[j+k],y=a[j+k+c]*(LL)G[k]%md;
            upd(a[j+k]+=y-md),upd(a[j+k+c]=x-y);
        }
    }
    if(!f){
        const int iv=pow(lim,md-2);
```

```
        for(int i=0;i<lim;++i)a[i]=(LL)a[i]*iv%md;
        std::reverse(a+1,a+lim);
    }
}
void work(){
    B1[0][0]=B2[0][0]=1;
    for(int i=0;i<m;++i)B1[1][i]=B[i];
    NTT(B,1);
    for(int i=2;i<=L;++i){
        int*Bp=B1[i-1],*Bn=B1[i];
        NTT(Bp,1);
        for(int i=0;i<lim;++i)Bn[i]=(LL)B[i]*Bp[i]%md;
        NTT(Bp,0),NTT(Bn,0);
        for(int i=n;i<n<<1;++i)Bn[i]=0;
    }
    for(int i=0;i<n;++i)B2[1][i]=B1[L][i];
    int*bL=B1[L];
    NTT(bL,1);
    for(int i=2;i<L;++i){
        int*Bp=B2[i-1],*Bn=B2[i];
        NTT(Bp,1);
        for(int i=0;i<lim;++i)Bn[i]=(LL)bL[i]*Bp[i]%md;
        NTT(Bp,0),NTT(Bn,0);
        for(int i=n;i<n<<1;++i)Bn[i]=0;
    }
    NTT(bL,0);
}
int main(){
    scanf("%d%d",&n,&m),++n,++m;
    for(int i=0;i<17;++i){
        int*G=g[i];
        G[0]=1;
        const int gi=G[1]=pow(3,(md-1)/(1<<i+1));
        for(int j=2;j<1<<i;++j)G[j]=(LL)G[j-1]*gi%md;
    }
    L=sqrt(n)+1;
    init(n<<1);
    for(int i=0;i<n;++i)scanf("%d",A+i);
    for(int i=0;i<m;++i)scanf("%d",B+i);
    work();
    for(int i=0;i<L;++i){
        static int C[N];
        for(int i=0;i<lim;++i)C[i]=0;
        for(int j=0;j<L;++j){
            const int x=A[i*L+j],*B=B1[j];
            for(int k=0;k<n;++k)
            C[k]=(C[k]+(LL)x*B[k])%md;
        }
        NTT(C,1),NTT(B2[i],1);
        for(int j=0;j<lim;++j)C[j]=(LL)C[j]*B2[i][j]%md;
        NTT(C,0);
        for(int i=0;i<n;++i)upd(R[i]+=C[i]-md);
    }
    for(int i=0;i<n;++i)
    printf("%d ",R[i]);
    return 0;
```

```
}
```

## 快速插值

首先根据拉格朗日插值公式我们可以暴力$O(n^2)$插出这个多项式，然而这显然是$gg$的

那么看看怎么优化，先来看一看拉格朗日插值的公式

$$f(x)=\sum_{i=1}^{n}y_i\prod_{i\neq j}\frac{x-x_j}{x_i-x_j}$$

转化一下

$$f(x)=\sum_{i=1}^{n}\frac{y_i}{\prod_{i\neq j}x_i-x_j}\prod_{i\neq j}(x-x_j)$$

来考虑一下$\frac{y_i}{\prod_{i\neq j}x_i-x_j}$这个东西，上面是个常数，那么只考虑下面。如果我们设多项式$g(x)=\prod_{i=1}^{n}(x-x_i)$，那么下面那个东西就是$\frac{g(x_i)}{(x-x_i)}$

这分子分母全为0我怎么求啊.....

根据~~我也不知道是啥的~~洛必达法则，如果

$$\lim_{x\to a}f(x)=0,\lim_{x\to a}g(x)=0$$

则有

$$\lim_{x\to a}\frac{f(x)}{g(x)}=\lim_{x\to a}\frac{f'(x)}{g'(x)}$$

那么我们代入之后可以发现$\frac{g(x_i)}{(x-x_i)}=g'(x_i)$

先分治$NTT$算出$g$，然后多点求值把每个点处的值算出来就好了

那么接下来我们就考虑分治，设$f_{l,r}$表示$(x_l,y_l),(x_r,y_r)$这些点算出来的答案，则有

$$\begin{aligned}f_{l,r}&=\sum_{i=l}^{r}\frac{y_i}{g'(x_i)}\prod_{j=l,i\neq j}^{r}(x-x_j)\\&=\prod_{j=mid+1}^{r}(x-x_j)\sum_{i=l}^{mid}\frac{y_i}{g'(x_i)}\prod_{j=l,i\neq j}^{mid}(x-x_j)+\prod_{j=l}^{mid}(x-x_j)\sum_{i=mid+1}^{r}\frac{y_i}{g'(x_i)}\prod_{j=mid+1,i\neq j}^{r}(x-x_j)\\&=\prod_{j=mid+1}^{r}(x-x_j)f_{l,mid}+\prod_{j=l}^{mid}(x-x_j)f_{mid+1,r}\end{aligned}$$

然后没有然后了 复杂度为$O(n\log^2 n)$

---

```
#include <cstdio>
#include <cstring>

const int maxn = 3e6+5 , mod = 998244353;

template<typename T>
inline T max(const T &a,const T &b){
    return a>b?a:b;
}
```

```
template<typename T>
inline void swap(T &a,T &b){
    T temp=a;a=b;b=temp;
}

struct IO{
    IO(){};char c;
    inline char gc(){
        static char buf[maxn],*p1=buf,*p2=buf;
        return p1==p2&&(p2=(p1=buf)+fread(buf,1,maxn,stdin),p1==p2)?EOF:*p1++;
    }
    template<typename T>
    inline IO&operator>>(T &_){
        _=0;bool f=1;c=gc();while(c<'0'||c>'9'){if(c=='-') f=0; c=gc();}
        while(c>='0'&&c<='9'){_=_*10+c-48;c=gc();}if(!f) _=-_;return *this;
    }
    char buf[maxn];int p = 0;~IO(){fwrite(buf,1,p,stdout);}
    inline void pc(const char &c){
        buf[p++] = c;
        if(p==maxn) fwrite(buf,1,maxn,stdout),p=0;
    }
    template<typename T>
    inline IO&operator<<(T x){
        if(!x){pc(48);return *this;}static int wt[41],len;len=0;if(x<0){pc('-
');x=-x;}
        for(;x;x/=10){wt[++len]=x%10;}
        while(len){pc(wt[len--]+48);}return *this;
    }
    inline IO&operator<<(const char &c){pc(c);return *this;}
}io;

int fastpow(int x,int y){
    if(y==0) return 1;int tmp=fastpow(x,y>>1);
    return y&1?1ll*tmp*tmp%mod*x%mod:1ll*tmp*tmp%mod ;
}

inline int add(int x,int y){return x+y>=mod?x+y-mod:x+y;}
inline int sub(int x,int y){return x-y<0?x-y+mod:x-y;}

inline void copy(int *Alpha,int *Beta,int len){
    //Alpha -> Beta
    for(int i=0;i<len;i++) Alpha[i]=Beta[i];
}

namespace Poly{
    const int gate = 3,invg = fastpow(gate,mod-2);
    int lim,lg,sing[maxn];int Gate[maxn],Invg[maxn];int inv[maxn];
    void init(int len){
        lim = 1,lg = 0;while(lim<len) lim<<=1,lg++;
        for(int i=1;i<lim;i++) sing[i]=(sing[i>>1]>>1)|((i&1)<<(lg-1));
    }
    int Hacking_to_the(){
        for(int i=1;i<maxn;i<<=1) Gate[i] = fastpow(gate,(mod-1)/(i<<1)),Invg[i]
= fastpow(invg,(mod-1)/(i<<1));
        // inv[1] = 1 ;for(int i=2;i<maxn;i++) inv[i] = (mod-1ll*
(mod/i)*inv[mod%i]%mod) ;
```

```
        return 0 ;
    }
    int GATE = Hacking_to_the();
    void NTT(int *Steins,int type){
        for(int i=0;i<lim;i++) if(i<sing[i]) swap(Steins[i],Steins[sing[i]]);
        for(int len=1;len<lim;len<<=1){
            int unit = type==1?Gate[len]:Invg[len];
            for(int i=0;i<lim;i+=(len<<1)){
                int w = 1;
                for(int k=0;k<len;k++,w=1ll*w*unit%mod){
                    int x=Steins[i+k],y=1ll*w*Steins[i+k+len]%mod;
                    Steins[i+k]=add(x,y),Steins[i+k+len]=sub(x,y);
                }
            }
        }
        if(type!=1){int Okabe=fastpow(lim,mod-2);for(int i=0;i<lim;i++)
Steins[i]=1ll*Steins[i]*Okabe%mod;}
    }
    void convolution(int *Alpha,int la,int *Beta,int lb,int *Zeta){
        init(la+lb-1);for(int i=la;i<lim;i++) Alpha[i]=0;for(int i=lb;i<lim;i++)
Beta[i]=0;
        NTT(Alpha,1),NTT(Beta,1);
        for(int i=0;i<lim;i++) Zeta[i]=1ll*Alpha[i]*Beta[i]%mod;return
NTT(Zeta,-1) ;
    }
    int g[maxn];
    void get_inv(int *Alpha,int len,int *Beta){
        //使用前清空Beta数组
        if(len==1){Beta[0]=fastpow(Alpha[0],mod-2);return ;}
        get_inv(Alpha,(len+1)>>1,Beta);init(len+len-1);
        copy(g,Alpha,len);for(int i=len;i<lim;i++) g[i] = 0;
        NTT(Beta,1),NTT(g,1);
        for(int i=0;i<lim;i++) Beta[i]=1ll*Beta[i]*(mod+2-
1ll*Beta[i]*g[i]%mod)%mod;
        NTT(Beta,-1);
        for(int i=len;i<lim;i++) Beta[i]=0;
    }
    const int inv2=fastpow(2,mod-2);
    int t[maxn];
    void get_sqrt(int *Alpha,int len,int *Beta){
        //使用前清空Beta数组
        if(len==1){Beta[0]=1;return ;}get_sqrt(Alpha,(len+1)>>1,Beta);
        get_inv(Beta,len,t);
        init(len+len-1);copy(g,Alpha,len);for(int i=len;i<lim;i++) g[i]=0;
        convolution(g,len,t,len,g);
        for(int i=0;i<lim;i++) Beta[i]=1ll*inv2*(Beta[i]+g[i])%mod;
        for(int i=len;i<lim;i++) Beta[i]=0;for(int i=0;i<lim;i++) t[i]=0;
    }
    void derivative(int *Alpha,int len,int *Beta){
        for(int i=1;i<len;i++) Beta[i-1]=1ll*i*Alpha[i]%mod;
        Beta[len-1] = 0;
    }
    void integral(int *Alpha,int len,int *Beta){
        for(int i=len-1;i>=1;i--) Beta[i]=1ll*Alpha[i-1]*inv[i];
        Beta[0] = 0;
    }
```

```
    int f1[maxn],f2[maxn];
    void get_ln(int *Alpha,int len,int *Beta){
        memset(f2,0,sizeof(f2));
        derivative(Alpha,len,f1),get_inv(Alpha,len,f2);
        convolution(f1,len,f2,len,Beta);
        return integral(Beta,len,Beta) ;
    }
    int e1[maxn],e2[maxn];
    void get_exp(int *Alpha,int len,int *Beta){
        //使用前清空Beta数组
        if(len==1) return void(Beta[0]=1);
        get_exp(Alpha,(len+1)>>1,Beta);get_ln(Beta,len,e1);
        init(len+len-1);
        for(int i=0;i<len;i++) e2[i] = sub(Alpha[i],e1[i]);for(int
i=len;i<lim;i++) e2[i]=0;
        NTT(e2,1),NTT(Beta,1);
        for(int i=0;i<lim;i++) Beta[i]=1ll*Beta[i]*(e2[i]+1)%mod;
        NTT(Beta,-1);
        for(int i=len;i<lim;i++) Beta[i]=0;for(int i=0;i<lim;i++) e1[i]=0;
    }
    int g1[maxn],g2[maxn],g3[maxn];
    void division(int *Alpha,int la,int *Beta,int lb,int *Zeta,int *Gamma){
        //Zeta's length requires 2la-lb
        if(la<lb){
            for(int i=0;i<=la-lb;i++) Zeta[i] = 0;
            for(int i=la;i<lb;i++) Gamma[i] = 0;return copy(Gamma,Alpha,la);
        }
        for(int i=0;i<la;i++) g1[i]=Alpha[la-i-1];for(int i=0;i<lb;i++)
g2[i]=Beta[lb-i-1];
        for(int i=lb;i<=la-lb;i++) g2[i]=0;
        lim=1;while(lim<=(la-lb)<<1) lim<<=1;for(int i=0;i<lim;i++) g3[i]=0;
        get_inv(g2,la-lb+1,g3);
        convolution(g1,la,g3,la-lb+1,Zeta);for(int i=0;i<la-lb-i;i++)
swap(Zeta[i],Zeta[la-lb-i]);
        for(int i=la-lb+1;i<lim;i++) Zeta[i]=0;
        copy(g1,Beta,lb);copy(g2,Zeta,la-lb+1);
        convolution(g1,lb,g2,la-lb+1,g1);for(int i=0;i<lb;i++)
Gamma[i]=sub(Alpha[i],g1[i]);
    }
}

namespace Multi_point{
    int *gate[maxn<<2];
    void init(int now,int l,int r,int *Alpha){
        if(l==r){gate[now] = new int[2];gate[now][0]=mod-Alpha[l],gate[now]
[1]=1;return ;}
        int mid = (l+r) >> 1,lim = 1;while(lim<r-l+1+1) lim<<=1;
        gate[now] = new int[lim];
        int *g1 = new int[lim],*g2 = new int[lim];
        init(now<<1,l,mid,Alpha),init(now<<1|1,mid+1,r,Alpha);
        copy(g1,gate[now<<1],mid-l+1+1);
        copy(g2,gate[now<<1|1],r-mid+1);
        return Poly::convolution(g1,mid-l+1+1,g2,r-mid+1,gate[now]) ;
    }
    void evaluation(int now,int l,int r,int *Alpha,int *Beta,int *Zeta){
        //Alpha->f(x),Beta->S{x}
```

```
        if(r-l+1<=256){
            for(int i=l;i<=r;i++){
                int answer = 0;int base = 1;
                for(int j=0;j<=r-l;j++,base=1ll*base*Beta[i]%mod)
                answer = (answer + 1ll*base*Alpha[j]) % mod;
                Zeta[i] = answer;
            }
            return ;//Which makes it faster;
        }
        int mid = (l+r) >> 1,lim = 1;while(lim<r-l+1+1) lim<<=1;
        int *g1,*g2;g1 = new int[lim<<1],g2 = new int[lim];
        Poly::division(Alpha,r-l+1,gate[now<<1],mid-l+1+1,g1,g2);
        evaluation(now<<1,l,mid,g2,Beta,Zeta);
        Poly::division(Alpha,r-l+1,gate[now<<1|1],r-mid+1,g1,g2);
        evaluation(now<<1|1,mid+1,r,g2,Beta,Zeta);
        delete[] g1,delete[] g2;
    }
    int g1[maxn],g2[maxn];
    void Evaluation(int *Alpha,int len,int *Beta,int n,int *Zeta){
        init(1,1,n,Beta);Poly::division(Alpha,len,gate[1],n+1,g1,g2);
        return evaluation(1,1,n,g2,Beta,Zeta) ;
    }
    int f1[maxn],f2[maxn],ell[maxn];
    int *Steins[maxn<<1],e1[maxn],e2[maxn],G1[maxn],G2[maxn];
    void interpolation(int now,int l,int r){
        if(l==r){
            Steins[now] = new int[1];
            return void(Steins[now][0] = ell[l]);
        }
        Steins[now] = new int[r-l+1];
        int mid = (l+r) >> 1,lim = 1;
        interpolation(now<<1,l,mid),interpolation(now<<1|1,mid+1,r);
        while(lim<(r-l+1)) lim<<=1;
        copy(G1,Steins[now<<1],mid-l+1),copy(G2,Steins[now<<1|1],r-mid);
        copy(e1,gate[now<<1|1],r-mid+1),copy(e2,gate[now<<1],mid-l+1+1);
        Poly::convolution(e1,r-mid+1,G1,mid-l+1,e1);
        Poly::convolution(e2,mid-l+1+1,G2,r-mid,e2);
        for(int i=0;i<=r-l;i++) Steins[now][i]=(e1[i]+e2[i])%mod;
    }
    void Interpolation(int *Alpha,int *Beta,int n,int *Zeta){
        //Alpha->x,Beta->y
        init(1,1,n,Alpha);copy(f1,gate[1],n+1);Poly::derivative(f1,n+1,f2);
        evaluation(1,1,n,f2,Alpha,ell);
        for(int i=1;i<=n;i++) ell[i]=1ll*Beta[i]*fastpow(ell[i],mod-2)%mod;
        interpolation(1,1,n);
        return copy(Zeta,Steins[1],n) ;
    }
}

int n ;int Alpha[maxn],Beta[maxn],Zeta[maxn];

int main(){
    scanf("%d",&n);for(int i=1;i<=n;i++) scanf("%d%d",&Alpha[i],&Beta[i]);
    Multi_point::Interpolation(Alpha,Beta,n,Zeta);
    for(int i=0;i<n;i++) io << Zeta[i] << " \n"[i==n-1];
    return 0;
```

```
}
```

### 常系数线性递推

$$\frac{F}{G}(k)$$

```
int divAt(Poly F,Poly G, ll k){
    int i;
    for(;k;k>>=1){
        Poly R=G;
        // R=G(-x)
        for(i=1;i<R.size();i+=2)R[i]=mod-R[i];
        F*=R,G*=R;
        for(i=k&1;i<F.size();i+=2)F[i/2]=F[i];
        F.resize(i/2);
        for(i=0;i<G.size();i+=2)G[i/2]=G[i];
        G.resize(i/2);
    }
    return F.empty()?0:F[0]*qpow(G[0],mod-2)%mod;
}
```

# 几何

## 凸包

```
int sgn( const double& x ) {
    return fabs(x) <= eps ? 0 : (x < 0 ? -1 : 1);
}
struct Point {
    double x , y; int id;
    Point( double x = 0 , double y = 0 ) : x(x) , y(y) {}
};
inline Point operator + ( const Point& a , const Point& b ) {
    return Point( a.x + b.x , a.y + b.y );
}
inline Point operator - ( const Point& a , const Point& b ) {
    return Point( a.x - b.x , a.y - b.y );
}
inline Point operator * ( const Point& a , const double& b ) {
    return Point( a.x * b , a.y * b );
}
inline Point operator / ( const Point& a , const double& b ) {
    return Point( a.x / b , a.y / b );
}
inline bool operator < ( const Point& a , const Point& b ) { // X first , Y
second
    return !sgn( a.x - b.x ) ? sgn( a.y - b.y ) < 0 : sgn( a.x - b.x ) < 0;
}
//inline bool operator < ( const Point& a , const Point& b ) { // Y first , X
second
//    return !sgn( a.y - b.y ) ? sgn( a.x - b.x ) < 0 : sgn( a.y - b.y ) < 0;
```

```
//}
inline db dot( ct Point& a , ct Point& b ) {
    return a.x * b.x + a.y * b.y;
}
inline db det( ct Point& a , ct Point& b ) {
    return a.x * b.y - a.y * b.x;
}
inline bool cross( ct Point& a , ct Point& b , ct Point& c , ct Point& d ) {
    return ( min( a.x , b.x ) <= max( c.x , d.x ) &&
        min( c.x , d.x ) <= max( a.x , b.x ) &&
        min( a.y , b.y ) <= max( c.y , d.y ) &&
        min( c.y , d.y ) <= max( a.y , b.y ) &&
        det( c - a , b - a ) * det( d - a , b - a ) <= 0 &&
        det( a - c , d - c ) * det( b - c , d - c ) <= 0 );
}
double dist( Point& a , Point& b ) {
    return sqrt( ( a.x - b.x ) * ( a.x - b.x ) + ( a.y - b.y ) * ( a.y - b.y ) );
}
Point _P;
bool compareForPolarAngle( Point a , Point b ) {
    double t = det( a - _P , b - _P );
    if( sgn( t ) == 0 ) return dist( a , _P ) > dist( b , _P );
    return sgn( t ) > 0;
}
Point crossPoint( Point lx , Point ly , Point rx , Point ry ) {
    double dd = det( lx - ly , ly - ry ) / det( lx - ly , rx - ry );
    return ( rx - ry ) * dd + ry; // ??
}
bool sameLine( Point a , Point b , Point c ) {
    return sgn( det( b - a , c - a ) ) == 0;
}
int top , stk[MAXN];
vector<Point> convexHull( vector<Point> A ) {
    if( A.size() == 1 ) return A;
    sort( A.begin() , A.end() );
    int n = A.size() , cur = -1;
    vector<Point> ret( 2 * n );
    rep( i , 0 , n - 1 ) {
        while( cur > 0 && sgn( det( ret[cur] - ret[cur - 1] , A[i] - ret[cur - 1]
) ) <= 0 ) -- cur;
        ret[++ cur] = A[i];
    }
    int t = cur;
    per( i , n - 2 , 0 ) {
        while( cur > t && sgn( det( ret[cur] - ret[cur - 1] , A[i] - ret[cur - 1]
) ) <= 0 ) -- cur;
        ret[++ cur] = A[i];
    }
    ret.resize( cur );// delete 0
    return ret;
}
```

# 半平面交

```
inline bool segmentCross( ct Point& a , ct Point& b , ct Point& c , ct Point& d )
{
    return ( min( a.x , b.x ) <= max( c.x , d.x ) &&
             min( c.x , d.x ) <= max( a.x , b.x ) &&
             min( a.y , b.y ) <= max( c.y , d.y ) &&
             min( c.y , d.y ) <= max( a.y , b.y ) &&
             det( c - a , b - a ) * det( d - a , b - a ) <= 0 &&
             det( a - c , d - c ) * det( b - c , d - c ) <= 0 );
}
db dist( Point& a , Point& b ) {
    return sqrt( ( a.x - b.x ) * ( a.x - b.x ) + ( a.y - b.y ) * ( a.y - b.y ) );
}
Point _P( 0 , 0 );
bool compareForPolarAngle( Point a , Point b ) {
    db t = det( a - _P , b - _P );
    if( sgn( t ) == 0 ) return dist( a , _P ) > dist( b , _P );
    return sgn( t ) > 0;
}
Point crossPoint( Point lx , Point ly , Point rx , Point ry ) {
    db dd = det( lx - ly , ly - ry ) / det( lx - ly , rx - ry );
    return ( rx - ry ) * dd + ry; // 交点
}
//bool sameLine( Point a , Point b , Point c ) {
//    return sgn( det( b - a , c - a ) ) == 0;
//}

double angle( const Point& x ) {
    return atan2( x.y , x.x );
}

struct Plane {
    Point p0 , p1;
    double ang;
    Plane( ) : p0(0) , p1(0) , ang(0) {}
    Plane( double a , double b , double c , double d ) {
        p0 = Point( a , b ) , p1 = Point( c , d );
        ang = angle( p1 - p0 );
    }
    Plane( const Point& a , const Point& b ) : p0( a ) , p1( b ) { ang = angle(
p1 - p0 ); }
    inline bool operator < ( const Plane& t ) const {
        int s = sgn( ang - t.ang );
        if( s ) return s == -1;
        else return sgn( det( p1 - p0 , t.p1 - p0 ) ) == -1;
    }
} L[MAXN] ;

inline bool parallel( const Plane& a , const Plane& b ) {
    return sgn( det( a.p1 - a.p0 , b.p1 - b.p0 ) == 0 );
}

inline bool invalid( const Plane& a , const Point& b ) {
    return sgn( det( a.p1 - a.p0 , b - a.p0 ) ) == -1;
```

```
}

Point crossPoint( Plane a , Plane b ) {
    return crossPoint( a.p0 , a.p1 , b.p0 , b.p1 );
}

Point ret[MAXN]; int tp;
Plane Q[MAXN]; Point qp[MAXN]; int hd , tl;
bool SI( Plane A[] , int n ) {
    hd = 0 , tl = -1;
    sort( A + 1 , A + 1 + n );
    Q[++ tl] = A[1];
    rep( i , 2 , n ) if( sgn( A[i].ang - A[i - 1].ang ) ) {
        if( hd < tl && ( parallel( Q[hd] , Q[hd + 1] ) || parallel( Q[tl] , Q[tl
- 1] ) ) ) return false;
        while( hd < tl && invalid( A[i] , qp[tl - 1] ) ) -- tl;
        while( hd < tl && invalid( A[i] , qp[hd] ) ) ++ hd;
        Q[++ tl] = A[i];
        if( hd < tl ) qp[tl - 1] = crossPoint( Q[tl] , Q[tl - 1] );
    }
    while( hd < tl && invalid( Q[hd] , qp[tl - 1] ) ) -- tl;
    while( hd < tl && invalid( Q[tl] , qp[hd] ) ) ++ hd;
    if( tl - hd <= 1 ) return false;
    qp[tl] = crossPoint( Q[tl] , Q[hd] );
    rep( i , hd , tl ) ret[++ tp] = qp[i];
    return true;
}

Point A[MAXN]; Plane fk[MAXN]; int sz;
void solve() {
    cin >> n;
    rep( i , 1 , n ) {
        scanf("%d",&m);
        rep( j , 1 , m )
            scanf("%lf%lf",&A[j].x,&A[j].y);
//      if( sgn( det( A[2] - A[1] , A[3] - A[1] ) ) == 1 ) {
            rep( j , 1 , m - 1 ) fk[++sz] = Plane( A[j] , A[j + 1] );
            fk[++ sz] = Plane( A[m] , A[1] );
//      }
//      else {
//          rep( j , 1 , m - 1 ) fk[++sz] = Plane( A[j + 1] , A[j] );
//          fk[++ sz] = Plane( A[1] , A[m] );
//      }
    }
    if( !SI( fk , sz ) ) return puts("0.000") , void();
    double res = 0;
    rep( i , 1 , tp - 1 )
        res += det( ret[i] , ret[i + 1] );
    res += det( ret[tp] , ret[1] );
    printf("%.3lf",res / 2);
}
```

# 旋转卡壳

凸包直径

```
vector<Point> convexHull( vector<Point>& A ) {
    int n = A.size();
    if( n == 1 ) return A;
    sort( all( A ) );
    vector<Point> r( n * 2 );
    int cur = -1;
    rep( i , 0 , n - 1 ) {
        while( cur > 0 && sgn( det( r[cur] - r[cur - 1] , A[i] - r[cur - 1] ) )
<= 0 ) -- cur;
        r[++ cur] = A[i];
    }
    int t = cur;
    per( i , n - 2 , 0 ) {
        while( cur > t && sgn( det( r[cur] - r[cur - 1] , A[i] - r[cur - 1] ) )
<= 0 ) -- cur;
        r[++ cur] = A[i];
    }
    r.resize( cur );
    return r;
}

double dis( Point a , Point b , Point c ) { // dis from c to ab
    return ( 1. * abs( det( b - a , c - a ) ) / ( sqrt( dis( a , b ) ) ) );
}

void solve() {
    int n;
    cin >> n;
    vector<Point> A( n );
    rep( i , 0 , n - 1 ) {
        int x , y;
        scanf("%d%d",&x,&y);
        A[i] = Point( x , y );
    }
    vector<Point> S = convexHull( A );
    n = S.size();
    int cur = 2;
    db as = 0;
    if( n == 2 ) {
        printf("%lld\n",dis( S[0] , S[1] ) );
        return;
    }
    rep( i , 0 , n - 1 ) {
        int nx = ( i + 1 ) % n;
        while( dis( S[i] , S[nx] , S[cur] ) < dis( S[i] , S[nx] , S[( cur + 1 ) %
n] ) ) cur = ( cur + 1 ) % n;
//      cout << S[i].x << ' ' << S[i].y << ' ' << S[cur].x << ' ' << S[cur].y <<
endl;
        as = max( as , max( dis( S[cur] , S[i] ) , dis( S[cur] , S[nx] ) ) );
    }
    printf("%lld\n",as);
```

```
}
```

## 三角形外接圆

过 $xOy$ 平面不共线三点 $M_i(x_i,y_i)$，$i=1,2,3$ 的圆 $\odot P$ 曲线方程为：

$$\begin{vmatrix} x^2+y^2 & x & y & 1 \\ {x_1}^2+{y_1}^2 & x_1 & y_1 & 1 \\ {x_2}^2+{y_2}^2 & x_2 & y_2 & 1 \\ {x_3}^2+{y_3}^2 & x_3 & y_3 & 1 \end{vmatrix} = 0$$

而圆心 $P$ 的坐标（按圆曲线方程中行列式第一行展开可得）为：

$$x_P = \frac{\begin{vmatrix} \frac{{x_1}^2+{y_1}^2}{2} & y_1 & 1 \\ \frac{{x_2}^2+{y_2}^2}{2} & y_2 & 1 \\ \frac{{x_3}^2+{y_3}^2}{2} & y_3 & 1 \end{vmatrix}}{\begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}} \qquad y_P = \frac{\begin{vmatrix} x_1 & \frac{{x_1}^2+{y_1}^2}{2} & 1 \\ x_2 & \frac{{x_2}^2+{y_2}^2}{2} & 1 \\ x_3 & \frac{{x_3}^2+{y_3}^2}{2} & 1 \end{vmatrix}}{\begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}}$$

$$y_P = \frac{\begin{vmatrix} 1 & y_1 & z_1 \\ 1 & y_2 & z_2 \\ 1 & y_3 & z_3 \end{vmatrix} \begin{vmatrix} 1 & \frac{{x_1}^2+{y_1}^2+{z_1}^2}{2} & z_1 \\ 1 & \frac{{x_2}^2+{y_2}^2+{z_2}^2}{2} & z_2 \\ 1 & \frac{{x_3}^2+{y_3}^2+{z_3}^2}{2} & z_3 \end{vmatrix} + \begin{vmatrix} x_1 & 1 & z_1 \\ x_2 & 1 & z_2 \\ x_3 & 1 & z_3 \end{vmatrix} \begin{vmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{vmatrix} + \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix} \begin{vmatrix} x_1 & \frac{{x_1}^2+{y_1}^2+{z_1}^2}{2} & 1 \\ x_2 & \frac{{x_2}^2+{y_2}^2+{z_2}^2}{2} & 1 \\ x_3 & \frac{{x_3}^2+{y_3}^2+{z_3}^2}{2} & 1 \end{vmatrix}}{\begin{vmatrix} 1 & y_1 & z_1 \\ 1 & y_2 & z_2 \\ 1 & y_3 & z_3 \end{vmatrix}^2 + \begin{vmatrix} x_1 & 1 & z_1 \\ x_2 & 1 & z_2 \\ x_3 & 1 & z_3 \end{vmatrix}^2 + \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}^2}$$

# 杂项

## 高斯消元

### 解方程

```
#define MAXN 106
#define eps 1e-8
int n;
double A[MAXN][MAXN];
int main() {
    cin >> n;
    int c;
    for( int i = 1 ; i <= n ; ++ i )
```

```
        for( int j = 1 ; j <= n + 1 ; ++ j )
            scanf("%d",&c) , A[i][j] = c;
    for( int i = 1 ; i <= n ; ++ i ) { // cur : 第 i 行
        int mxpos = i;
        for( int j = i ; j <= n ; ++ j ) if( fabs( A[j][i] ) - fabs( A[mxpos][i]
) > eps )
                mxpos = j;
        swap( A[i] , A[mxpos] );
        double c = A[i][i];
        if( c == 0 ) { puts("No Solution"); return 0; }
        A[i][i] = 1;
        for( int j = i + 1 ; j <= n + 1 ; ++ j )
            A[i][j] /= c;
        for( int j = 1 ; j <= n ; ++ j ) if( j != i ) { // cur : 第 j 行
                double cc = A[j][i]; A[j][i] = 0;
                for( int k = i + 1 ; k <= n + 1 ; ++ k ) // cur : 第 k 列
                    A[j][k] -= cc * A[i][k];
            }
    }
    for( int i = 1 ; i <= n ; ++ i ) {
        printf("%.2lf\n",A[i][n+1]);
    }
}
```

## 矩阵求逆

```
int A[MAXN][MAXN] , B[MAXN][MAXN];
int Pow( int x , int a ) {
    int ret = 1;
    while( a ) {
        if( a & 1 ) ret = ret * 1ll * x % P;
        x = x * 1ll * x % P , a >>= 1;
    }
    return ret;
}

void swap( int& a , int& b ) {
    a ^= b ^= a ^= b;
}

bool fuck( ) {
    for( int i = 1 ; i <= n ; ++ i ) {
        if( !A[i][i] ) {
            for( int j = i + 1 ; j <= n ; ++j ) if( A[j][i] ) {
                rep( k , 1 , n ) swap( A[i][k] , A[j][k] ) , swap( B[i][k] , B[j]
[k] );
                break;
            }
        }
        if( !A[i][i] ) return false;
        int iv = Pow( A[i][i] , P - 2 ); A[i][i] = 1;
        rep( j , i + 1 , n ) A[i][j] = A[i][j] * 1ll * iv % P;
        rep( j , 1 , n ) B[i][j] = B[i][j] * 1ll * iv % P;
        for( int j = 1 ; j <= n ; ++ j ) if( j != i && A[j][i] ) {
            int cc = A[j][i]; A[j][i] = 0;
```

```
            rep( k , 1 , n )
                B[j][k] = ( B[j][k] - cc * 1ll * B[i][k] % P ) , B[j][k] = B[j]
[k] < 0 ? B[j][k] + P : B[j][k];
            rep( k , i + 1 , n )
                A[j][k] = ( A[j][k] - cc * 1ll * A[i][k] % P ) , A[j][k] = A[j]
[k] < 0 ? A[j][k] + P : A[j][k];
        }
    }
    return true;
}
```

## 行列式

```
int Pow( int x , int a ) {
    int ret = 1;
    while( a ) {
        if( a & 1 ) ret = ret * 1ll * x % P;
        x = x * 1ll * x % P , a >>= 1;
    }
    return ret;
}

int det( int n ) {
    int ans = 1;
    rep( i , 1 , n ) rep( j , 1 , n ) G[i][j] = ( G[i][j] + P ) % P;
    rep( i , 1 , n ) {
        if( !G[i][i] ) {
            rep( j , i + 1 , n ) {
                if( G[j][i] ) {
                    swap( G[j] , G[i] ) , ans = P - ans; break;
                }
            }
        }
        ans = ans * 1ll * G[i][i] % P;
        rep( j , i + 1 , n ) if( G[j][i] ) {
            int kk = G[j][i] * 1ll * Pow( G[i][i] , P - 2 ) % P;
            rep( k , i , n )
                G[j][k] = ( G[j][k] + P - 1ll * kk * G[i][k] % P ) % P;
        }
    }
    return ans;
}
```

## FWT

$$FWT_{xor}(A)[i] = \sum_j A_j(-1)^{popcount(j\&i)}$$

```
namespace FWT {
#define inv2 ( ( P + 1 ) / 2 )
```

```
    inline void FWTor(int a[], int len) {
        for (int mid = 2; mid <= len; mid <<= 1)
            for (int i = 0; i < len; i += mid)
                for (int j = i; j < i + (mid >> 1); j++)
                    ( a[j + (mid >> 1)] += a[j] ) %= P;
    }
    inline void IFWTor(int a[], int len) {
        for (int mid = 2; mid <= len; mid <<= 1)
            for (int i = 0; i < len; i += mid)
                for (int j = i; j < i + (mid >> 1); j++)
                    ( a[j + (mid >> 1)] += P - a[j] ) %= P;
    }

    inline void FWTand(int a[], int len) {
        for (int mid = 2; mid <= len; mid <<= 1)
            for (int i = 0; i < len; i += mid)
                for (int j = i; j < i + (mid >> 1); j++)
                    ( a[j] += a[j + (mid >> 1)] ) %= P;
    }
    inline void IFWTand(int a[], int len) {
        for (int mid = 2; mid <= len; mid <<= 1)
            for (int i = 0; i < len; i += mid)
                for (int j = i; j < i + (mid >> 1); j++)
                    ( a[j] += P - a[j + (mid >> 1)] ) %= P;
    }

    inline void FWTxor(int a[], int len) {
        for (int mid = 2; mid <= len; mid <<= 1)
            for (int i = 0; i < len; i += mid)
                for (int j = i; j < i + (mid >> 1); j++) {
                    int x = a[j], y = a[j + (mid >> 1)];
                    a[j] = ( x + y ) % P, a[j + (mid >> 1)] = ( x - y + P ) % P;
                }
    }
    inline void IFWTxor(int a[], int len) {
        for (int mid = 2; mid <= len; mid <<= 1)
            for (int i = 0; i < len; i += mid)
                for (int j = i; j < i + (mid >> 1); j++) {
                    int x = a[j], y = a[j + (mid >> 1)];
                    a[j] = 1ll * ( x + y ) * inv2 % P , a[j + (mid >> 1)] = 1ll *
(x - y + P) * inv2 % P;
                }
    }
}
```

## 子集卷积

```
int A[1<<21] , B[1<<21] , n , len;
int a[21][1<<21] , b[21][1<<21] , c[21][1<<21];

void FMT( int A[] , int l ) {
    for( int i = 0 ; i < l ; ++ i )
        for( int j = 0 ; j < ( 1 << l ) ; ++ j )
            if( j & ( 1 << i ) ) A[j] = ( A[j] + A[j ^ ( 1 << i )] ) % P;
}
```

```
void IFMT( int A[] , int l ) {
    for( int i = 0 ; i < l ; ++ i )
        for( int j = 0 ; j < ( 1 << l ) ; ++ j )
            if( j & ( 1 << i ) ) A[j] = ( A[j] + P - A[j ^ ( 1 << i )] ) % P;
}

int main() {
    cin >> n; len = ( 1 << n );
    for( int i = 0 ; i < len ; ++ i ) A[i] = rd() , a[__builtin_popcount(i)][i] =
A[i];
    for( int i = 0 ; i < len ; ++ i ) B[i] = rd() , b[__builtin_popcount(i)][i] =
B[i];
    for( int i = 0 ; i <= n ; ++ i ) FMT( a[i] , n ) ,  FMT( b[i] , n );
    for( int x = 0 ; x <= n ; ++ x ) {
        for( int i = 0 ; i <= x ; ++ i )
            for( int j = 0 ; j < ( 1 << n ) ; ++ j )
                ( c[x][j] += 1ll * a[i][j] * b[x - i][j] % P ) %= P;
        IFMT( c[x] , n );
    }
    for( int i = 0 ; i < len ; ++ i ) printf("%d ",c[__builtin_popcount(i)][i]);
}
```

# Polya

## Burnside 定理

（烧边定理（？ 大雾））

对于一个置换 $f$，我们定义 $C(f)$ 为 在合成 $f$ 后保持相对于原来不变的方案数量，这里称作 不动点。**不动点并不是一个点，而是一个方案**！

以染色为例，“不动点”即指染色方案，满足合成 $f$ 后染色不变。

那么本质不同的方案数量就是对于左右可用置换不动点个数平均值，形式化地写：

$$ans = \frac{1}{|G|}\sum_{f\in G} C(f)$$

其中 $G$ 是一个置换群。

## Polya 定理

我们考虑知道了 Burnside 定理后如何求染色方案数量。

对于一个置换 $f$ 我们可以设 $L(f)$ 表示在 $i \to f_i$ 连边后的环的数量，那么不难发现一个置换是不动点，等价于所有的环内的位置是等价的。

也就是说，我们可以略微改写 Burside 定理

$$ans = \frac{1}{|G|}\sum_{f\in G} m^{L(f)}$$

其中 $m$ 为每个位置的方案数量，有时候 $m$ 也可以替换为其他东西，反正就是每个环必须全部等价就完事了。

这就是 Polya 定理辣。

```
int Pow( int x , int a ) {
    int ret = 1;
    while( a ) {
        if( a & 1 ) ret = 1ll * ret * x % P;
        x = 1ll * x * x % P , a >>= 1;
    }
    return ret;
```

```
}

int phi( int x ) {
    int res = x;
    for( int i = 2 ; i * i <= x ; ++ i ) if( x % i == 0 ) {
        res = res - res / i , x /= i;
        while( x % i == 0 ) x /= i;
    }
    if( x != 1 ) res = res - res / x;
    return res;
}

void solve() {
    scanf("%d",&n);
    int res = 0;
    for( int i = 1 ; i * i <= n ; ++ i ) if( n % i == 0 ) {
        ( res += 1ll * Pow( n , i ) * phi( n / i ) % P ) %= P;
        if( i * i != n ) ( res += 1ll * Pow( n , n / i ) * phi( i ) % P ) %= P;
    }
    printf("%d\n",1ll * res * Pow( n , P - 2 ) % P);
}
```

## 博弈

- 每次拿掉一个因子的 Nim： `__builtin_ctz(x)`
- N 阶 Nim 游戏：有 k 堆石子，各包含 $x_1,x_2...x_k$ 颗石子。双方玩家轮流操作，每次操作选择其中非空的若干堆，至少一堆但不超过 N 堆，在这若干堆中的每堆各取走其中的若干颗石子（1 颗，2 颗...甚至整堆），数目可以不同，取走最后一颗石子的玩家获胜。

  结论：当且仅当在每一个不同的二进制位上，$x_1,x_2...x_k$ 中在该位上 1 的个数是 N+1 的倍数时，后手方有必胜策略，否则先手必胜。
- 阶梯 Nim：把后面的石头放到前面
  看偶数堆的异或是否为 0，往前放 $k$ 类似，看 $\mod k+1$
- 每次删一条边和非根方向子树：儿子 sg + 1 异或起来
- 每次选一个叶子集合删掉：所有分叉长为偶数
- Nim 积：不会

## 辛普森积分

然后就得到了**Simpson公式**

$$\int_a^b f(x)dx \approx \frac{(b-a)(f(a)+f(b)+4f(\frac{a+b}{2}))}{6}$$

```
#include <bits/stdc++.h>
double a,b,c,d,l,r;
inline double f(double x) {
    return (c*x+d)/(a*x+b);        //原函数
}
```

```
inline double simpson(double l,double r) {      //Simpson公式
    double mid=(l+r)/2;
    return (f(l)+4*f(mid)+f(r))*(r-l)/6;
}
double asr(double l,double r,double eps,double ans) {
    double mid=(l+r)/2;
    double l_=simpson(l,mid),r_=simpson(mid,r);
    if(fabs(l_+r_-ans)<=15*eps) return l_+r_+(l_+r_-ans)/15;      //确认精度
    return asr(l,mid,eps/2,l_)+asr(mid,r,eps/2,r_);      //精度不够则递归调用
}
inline double asr(double l,double r,double eps) {
    return asr(l,r,eps,simpson(l,r));
}
int main() {
    scanf("%lf%lf%lf%lf%lf%lf",&a,&b,&c,&d,&l,&r);
    printf("%.6lf",asr(l,r,1e-6));
    return 0;
}
```

## Berlekamp–Massey

```
int Pow( int x , int a ) {
    int ans = 1 , cur = x % P;
    while( a ) {
        if( a & 1 ) ans = 1ll * ans * cur % P;
        cur = 1ll * cur * cur % P , a >>= 1;
    }
    return ans;
}

int del[MAXN];
vi BM( vi a ) {
    vi re , las;
    int fail = -1 , n = a.size();
    for( int i = 0 ; i < n ; ++ i ) {
        del[i] = a[i];
        for( int j = 0 ; j < re.size() ; ++ j ) del[i] = ( del[i] + P - a[i - j -
1] * 1ll * re[j] % P ) % P;
        if( !del[i] ) continue;
        if( fail == -1 ) { fail = i , re.resize( i + 1 ); continue; }
        int mu = del[i] * 1ll * Pow( del[fail] , P - 2 ) % P;
        vi tmp = re; re.clear() , re.resize( i - fail - 1 ) , re.pb( mu );
        for( int x : las ) re.pb( x * 1ll * ( P - mu ) % P );
        fail = i , las = tmp;
        if( re.size() < las.size() ) re.resize( tmp.size() );
        for( int i = 0 ; i < tmp.size() ; ++ i ) re[i] = ( re[i] + tmp[i] ) % P;
    }
    return re;
}
```

**一些不知道会不会有用的东西**

- $$\begin{aligned} ax+by&=e \\ cx+dy&=f \end{aligned} \Rightarrow \begin{aligned} x&=\frac{ed-bf}{ad-bc} \\ y&=\frac{af-ec}{ad-bc} \end{aligned}$$

- $$\frac{d}{dx}\arcsin x=\frac{1}{\sqrt{1-x^2}} \qquad \frac{d}{dx}\arccos x=-\frac{1}{\sqrt{1-x^2}}$$
$$\frac{d}{dx}\tan x=1+\tan^2 x \qquad \frac{d}{dx}\arctan x=\frac{1}{1+x^2}$$
$$\int \tan ax=-\frac{\ln|\cos ax|}{a} \qquad \int x\sin ax=\frac{\sin ax-ax\cos ax}{a^2}$$
$$\int e^{-x^2}=\frac{\sqrt{\pi}}{2}\mathrm{erf}(x) \qquad \int xe^{ax}dx=\frac{e^{ax}}{a^2}(ax-1)$$

- $$e^x=1+x+\frac{x^2}{2!}+\frac{x^3}{3!}+\ldots,\ (-\infty<x<\infty)$$
$$\ln(1+x)=x-\frac{x^2}{2}+\frac{x^3}{3}-\frac{x^4}{4}+\ldots,\ (-1<x\leq 1)$$
$$\sqrt{1+x}=1+\frac{x}{2}-\frac{x^2}{8}+\frac{2x^3}{32}-\frac{5x^4}{128}+\ldots,\ (-1\leq x\leq 1)$$
$$\sin x=x-\frac{x^3}{3!}+\frac{x^5}{5!}-\frac{x^7}{7!}+\ldots,\ (-\infty<x<\infty)$$
$$\cos x=1-\frac{x^2}{2!}+\frac{x^4}{4!}-\frac{x^6}{6!}+\ldots,\ (-\infty<x<\infty)$$

- HashMap.h

**Description:** Hash map with mostly the same API as unordered_map, but ~3x faster. Uses 1.5x memory. Initial capacity must be a power of 2 (if provided).

d77092, 7 lines

```
#include <bits/extc++.h>
// To use most bits rather than just the lowest ones:
struct chash { // large odd number for C
  const uint64_t C = ll(4e18 * acos(0)) | 71;
  ll operator()(ll x) const { return __builtin_bswap64(x*C); }
};
__gnu_pbds::gp_hash_table<ll,int,chash> h({},{},{},{},{1<<16});
```

- ## PolyRoots.h

**Description:** Finds the real roots to a polynomial.
**Usage:** `polyRoots({{2,-3,1}},-1e9,1e9) // solve x^2-3x+2 = 0`
**Time:** $\mathcal{O}\left(n^2 \log(1/\epsilon)\right)$

"Polynomial.h" b00bfe, 23 lines

```
vector<double> polyRoots(Poly p, double xmin, double xmax) {
  if (sz(p.a) == 2) { return {-p.a[0]/p.a[1]}; }
  vector<double> ret;
  Poly der = p;
  der.diff();
  auto dr = polyRoots(der, xmin, xmax);
  dr.push_back(xmin-1);
  dr.push_back(xmax+1);
  sort(all(dr));
  rep(i,0,sz(dr)-1) {
    double l = dr[i], h = dr[i+1];
    bool sign = p(l) > 0;
    if (sign ^ (p(h) > 0)) {
      rep(it,0,60) { // while (h - l > 1e-8)
        double m = (l + h) / 2, f = p(m);
        if ((f <= 0) ^ sign) l = m;
        else h = m;
      }
      ret.push_back((l + h) / 2);
    }
  }
  return ret;
}
```

- ## PolyInterpolate.h

**Description:** Given $n$ points (x[i], y[i]), computes an n-1-degree polynomial $p$ that passes through them: $p(x) = a[0] * x^0 + ... + a[n-1] * x^{n-1}$. For numerical precision, pick $x[k] = c * \cos(k/(n-1) * \pi), k = 0 \ldots n-1$.
**Time:** $\mathcal{O}\left(n^2\right)$

08bf48, 13 lines

```
typedef vector<double> vd;
vd interpolate(vd x, vd y, int n) {
  vd res(n), temp(n);
  rep(k,0,n-1) rep(i,k+1,n)
    y[i] = (y[i] - y[k]) / (x[i] - x[k]);
  double last = 0; temp[0] = 1;
  rep(k,0,n) rep(i,0,n) {
    res[i] += y[k] * temp[i];
    swap(last, temp[i]);
    temp[i] -= last * x[k];
  }
  return res;
}
```

- 线性规划

**Description:** Solves a general linear maximization problem: maximize $c^T x$ subject to $Ax \leq b$, $x \geq 0$. Returns -inf if there is no solution, inf if there are arbitrarily good solutions, or the maximum value of $c^T x$ otherwise. The input vector is set to an optimal $x$ (or in the unbounded case, an arbitrary solution fulfilling the constraints). Numerical stability is not guaranteed. For better performance, define variables such that $x = 0$ is viable.

**Usage:**
```
vvd A = {{1,-1}, {-1,1}, {-1,-2}};
vd b = {1,1,-4}, c = {-1,-1}, x;
T val = LPSolver(A, b, c).solve(x);
```

**Time:** $\mathcal{O}(NM * \#pivots)$, where a pivot may be e.g. an edge relaxation. $\mathcal{O}(2^n)$ in the general case.

aa8530, 68 lines

```
typedef double T; // long double, Rational, double + mod<P>...
typedef vector<T> vd;
typedef vector<vd> vvd;

const T eps = 1e-8, inf = 1/.0;
#define MP make_pair
#define ltj(X) if(s == -1 || MP(X[j],N[j]) < MP(X[s],N[s])) s=j

struct LPSolver {
  int m, n;
  vi N, B;
  vvd D;

  LPSolver(const vvd& A, const vd& b, const vd& c) :
    m(sz(b)), n(sz(c)), N(n+1), B(m), D(m+2, vd(n+2)) {
      rep(i,0,m) rep(j,0,n) D[i][j] = A[i][j];
      rep(i,0,m) { B[i] = n+i; D[i][n] = -1; D[i][n+1] = b[i];}
      rep(j,0,n) { N[j] = j; D[m][j] = -c[j]; }
      N[n] = -1; D[m+1][n] = 1;
    }

  void pivot(int r, int s) {
    T *a = D[r].data(), inv = 1 / a[s];
    rep(i,0,m+2) if (i != r && abs(D[i][s]) > eps) {
      T *b = D[i].data(), inv2 = b[s] * inv;
      rep(j,0,n+2) b[j] -= a[j] * inv2;
      b[s] = a[s] * inv2;
    }
```

```
    rep(j,0,n+2) if (j != s) D[r][j] *= inv;
    rep(i,0,m+2) if (i != r) D[i][s] *= -inv;
    D[r][s] = inv;
    swap(B[r], N[s]);
  }

  bool simplex(int phase) {
    int x = m + phase - 1;
    for (;;) {
      int s = -1;
      rep(j,0,n+1) if (N[j] != -phase) ltj(D[x]);
      if (D[x][s] >= -eps) return true;
      int r = -1;
      rep(i,0,m) {
        if (D[i][s] <= eps) continue;
        if (r == -1 || MP(D[i][n+1] / D[i][s], B[i])
                     < MP(D[r][n+1] / D[r][s], B[r])) r = i;
      }
      if (r == -1) return false;
      pivot(r, s);
    }
  }

  T solve(vd &x) {
    int r = 0;
    rep(i,1,m) if (D[i][n+1] < D[r][n+1]) r = i;
    if (D[r][n+1] < -eps) {
      pivot(r, n);
      if (!simplex(2) || D[m+1][n+1] < -eps) return -inf;
      rep(i,0,m) if (B[i] == -1) {
        int s = 0;
        rep(j,1,n+1) ltj(D[i]);
        pivot(i, s);
      }
    }
    bool ok = simplex(1); x = vd(n);
    rep(i,0,m) if (B[i] < n) x[B[i]] = D[i][n+1];
    return ok ? D[m][n+1] : inf;
  }
};
```

- ModSum.h

**Description:** Sums of mod'ed arithmetic progressions.
`modsum(to, c, k, m)` $= \sum_{i=0}^{to-1} (ki + c)\%m$. `divsum` is similar but for floored division.
**Time:** $\log(m)$, with a large constant. 5c5bc5, 16 lines

```
typedef unsigned long long ull;
ull sumsq(ull to) { return to / 2 * ((to-1) | 1); }

ull divsum(ull to, ull c, ull k, ull m) {
  ull res = k / m * sumsq(to) + c / m * to;
  k %= m; c %= m;
  if (!k) return res;
  ull to2 = (to * k + c) / m;
  return res + (to - 1) * to2 - divsum(to2, m-1 - c, m, k);
}

ll modsum(ull to, ll c, ll k, ll m) {
  c = ((c % m) + m) % m;
  k = ((k % m) + m) % m;
  return to * c + k * sumsq(to) - m * divsum(to, c, k, m);
}
```

- ### 6.3.1 Bernoulli numbers

EGF of Bernoulli numbers is $B(t) = \frac{t}{e^t - 1}$ (FFT-able).
$B[0, \ldots] = [1, -\frac{1}{2}, \frac{1}{6}, 0, -\frac{1}{30}, 0, \frac{1}{42}, \ldots]$

Sums of powers:

$$\sum_{i=1}^{n} n^m = \frac{1}{m+1} \sum_{k=0}^{m} \binom{m+1}{k} B_k \cdot (n+1)^{m+1-k}$$

Euler-Maclaurin formula for infinite sums:

$$\sum_{i=m}^{\infty} f(i) = \int_m^{\infty} f(x)dx - \sum_{k=1}^{\infty} \frac{B_k}{k!} f^{(k-1)}(m)$$

$$\approx \int_m^{\infty} f(x)dx + \frac{f(m)}{2} - \frac{f'(m)}{12} + \frac{f'''(m)}{720} + O(f^{(5)}(m))$$

- ## GeneralMatching.h

**Description:** Matching for general graphs. Fails with probability $N/mod$.
**Time:** $\mathcal{O}\left(N^3\right)$

"../numerical/MatrixInverse-mod.h" cb1912, 40 lines

```
vector<pii> generalMatching(int N, vector<pii>& ed) {
  vector<vector<ll>> mat(N, vector<ll>(N)), A;
  for (pii pa : ed) {
    int a = pa.first, b = pa.second, r = rand() % mod;
    mat[a][b] = r, mat[b][a] = (mod - r) % mod;
  }

  int r = matInv(A = mat), M = 2*N - r, fi, fj;
  assert(r % 2 == 0);

  if (M != N) do {
    mat.resize(M, vector<ll>(M));
    rep(i,0,N) {
      mat[i].resize(M);
      rep(j,N,M) {
        int r = rand() % mod;
        mat[i][j] = r, mat[j][i] = (mod - r) % mod;


      }
    }
  } while (matInv(A = mat) != M);

  vi has(M, 1); vector<pii> ret;
  rep(it,0,M/2) {
    rep(i,0,M) if (has[i])
      rep(j,i+1,M) if (A[i][j] && mat[i][j]) {
        fi = i; fj = j; goto done;
    } assert(0); done:
    if (fj < N) ret.emplace_back(fi, fj);
    has[fi] = has[fj] = 0;
    rep(sw,0,2) {
      ll a = modpow(A[fi][fj], mod-2);
      rep(i,0,M) if (has[i] && A[i][fj]) {
        ll b = A[i][fj] * a % mod;
        rep(j,0,M) A[i][j] = (A[i][j] - A[fi][j] * b) % mod;
      }
      swap(fi,fj);
    }
  }
  return ret;
}
```

- **Description:** Add and remove intervals from a set of disjoint intervals. Will merge the added interval with any overlapping intervals in the set when adding. Intervals are [inclusive, exclusive).
**Time:** $\mathcal{O}(\log N)$

edce47, 23 lines

```
set<pii>::iterator addInterval(set<pii>& is, int L, int R) {
  if (L == R) return is.end();
  auto it = is.lower_bound({L, R}), before = it;
  while (it != is.end() && it->first <= R) {
    R = max(R, it->second);
    before = it = is.erase(it);
  }
  if (it != is.begin() && (--it)->second >= L) {
    L = min(L, it->first);
    R = max(R, it->second);
    is.erase(it);
  }
  return is.insert(before, {L,R});
}

void removeInterval(set<pii>& is, int L, int R) {
  if (L == R) return;
  auto it = addInterval(is, L, R);
  auto r2 = it->second;
  if (it->first == L) is.erase(it);
  else (int&)it->second = L;
  if (R != r2) is.emplace(R, r2);
}
```

- ### 7.8.2 Erdős–Gallai theorem

A simple graph with node degrees $d_1 \geq \cdots \geq d_n$ exists iff $d_1 + \cdots + d_n$ is even and for every $k = 1 \ldots n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

开栈:

```
#pragma comment(linker,"/STACK:102400000,102400000")
-Wl,--stack=102400000
```

快读:

```
const int MAXSIZE= 1<<20;
char buf[MAXSIZE],*p1,*p2;
// #define gc() (p1==p2 && (p2=(p1=buf)+fread(buf,1,MAXSIZE,stdin),p1==p2)?
EOF:*p1++)
 #define gc() getchar()
inline int read(){
    int x=0;
    char c=gc();
    bool fs = 0;
    while(!isdigit(c)) fs |= ( c == '-' ) , c = gc();
    while(isdigit(c)) x=(x<<3)+(x<<1)+(c^48),c=gc();
    if( fs ) x = -x;
    return x;
}
```

```
namespace IO {
const int MAXSIZE = 1 << 20;
char buf[MAXSIZE], *p1, *p2;
#define gc()                                                                  \
  (p1 == p2 && (p2 = (p1 = buf) + fread(buf, 1, MAXSIZE, stdin), p1 == p2) \
       ? EOF                                                                  \
       : *p1++)
inline int rd() {
  int x = 0, f = 1;
  char c = gc();
  while (!isdigit(c)) {
    if (c == '-') f = -1;
    c = gc();
  }
  while (isdigit(c)) x = x * 10 + (c ^ 48), c = gc();
  return x * f;
}
char pbuf[1 << 20], *pp = pbuf;
inline void push(const char &c) {
  if (pp - pbuf == 1 << 20) fwrite(pbuf, 1, 1 << 20, stdout), pp = pbuf;
  *pp++ = c;
}
inline void write(int x) {
  static int sta[35];
  int top = 0;
  do {
    sta[top++] = x % 10, x /= 10;
  } while (x);
  while (top) push(sta[--top] + '0');
  push( 10 );
}
}
fwrite( pbuf , 1 , pp - pbuf , stdout );
```